TFI - BASES DE DATOS I

Sistema de Pedidos / Ventas Online

Materia: Bases de Datos I

Carrera: Tecnicatura Universitaria en Programación (TUP)

Comisión: C2 (Todos de comisión 2)

Grupos: Alejandro y Gabriel (G03), Josue (G12)

Integrantes:

Alejandro Fernández (M26 C2-03)

Gabriel Di Lorenzo (M26 C2-03)

Josue Gallardo (C2-12)

Fecha: Octubre 2026

ÍNDICE

1. Introducción y dominio
2. Etapa 1 — Modelo
3. Etapa 2 — Carga masiva
4. Etapa 3 — Consultas e índices
5. Etapa 4 — Seguridad e integridad
6. Etapa 5 — Transacciones y concurrencia
7. Conclusiones
8. Anexo IA
9. Referencias

---

1. INTRODUCCIÓN Y DOMINIO

1.1 Descripción del sistema

Este trabajo práctico integrador implementa un sistema de gestión de pedidos y ventas online, diseñado para una empresa de comercio electrónico. El sistema permite gestionar clientes, productos, categorías, pedidos y detalles de pedidos, con mecanismos de control de stock, procesamiento de ventas y reporting de ventas por categoría.

1.2 Alcance

El sistema cubre las siguientes funcionalidades principales: - Gestión de catálogo de productos organizados por categorías - Registro de clientes con datos de contacto - Procesamiento de pedidos con múltiples productos por pedido - Control de stock en tiempo real - Reporting de ventas por categoría y cliente - Gestión de seguridad con roles diferenciados - Control de concurrencia para operaciones de venta simultáneas

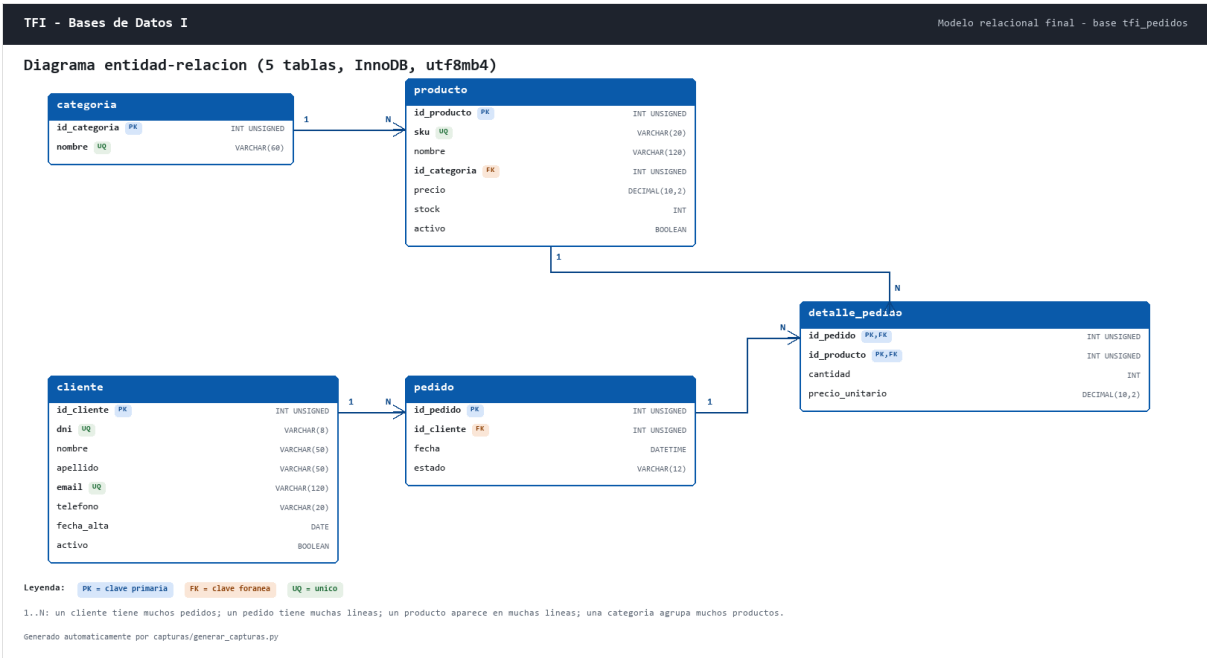
1.3 Tecnología

- **Motor de base de datos**: MySQL 8.0.42
- **Storage engine**: InnoDB (transaccional)
- **Codificación**: utf8mb4 (soporte Unicode completo)
- **Puerto**: 3307 (instancia portable)
- **Volumen de datos**: 345.010 registros distribuidos en 5 tablas

---

2. ETAPA 1 — MODELO

2.1 Diagrama Entidad-Relación (DER)



DER Final

El modelo relacional sigue la siguiente estructura:

```
categoria(1) ---- N producto(1) ---- N detalle_pedido(N) ---- N pedido(N) ---- 1 cliente
```

2.2 Modelo relacional

Tabla	Filas	Clave Primaria	Claves Foráneas
`categoria`	10	`id_categoria`	-
`cliente`	20.000	`id_cliente`	-
`producto`	5.000	`id_producto`	`fk_producto_categoria → categoria(id_categoria)`
`pedido`	80.000	`id_pedido`	`fk_pedido_cliente → cliente(id_cliente)`
`detalle_pedido`	240.000	`(id_pedido, id_producto)`	`fk_detalle_pedido → pedido(id_pedido)` `fk_detalle_producto → producto(id_producto)`

2.3 Decisiones de diseño y constraints

2.3.1 Precisión de tipos de datos

Todos los campos monetarios utilizan `DECIMAL(10,2)` para evitar errores de precisión de punto flotante: - `producto.precio` : DECIMAL(10,2) - `detalle_pedido.precio_unitario` : DECIMAL(10,2)

Las fechas se almacenan con tipos apropiados: - `pedido.fecha` : DATETIME (timestamp exacto del pedido) - `cliente.fecha_alta` : DATE (solo fecha de registro)

2.3.2 Constraint de CHECK en estado de pedido

El campo `estado` de la tabla `pedido` se restringe a 5 valores válidos:

```
CONSTRAINT chk_pedido_estado
CHECK (estado IN ('PENDIENTE','PAGADO','ENVIADO','ENTREGADO','CANCELADO'))
```

Esta restricción evita estados inválidos como NULL o valores no reconocidos.

2.3.3 Unique key en nombre de categoría

Se implementó una restricción UNIQUE en `categoria.nombre` para evitar duplicados semánticos:

```
CONSTRAINT uq_categoria_nombre UNIQUE (nombre)
```

Esto previene categorías con el mismo nombre pero diferente ID, que causarían confusión en reporting.

2.3.4 Acciones referenciales

Todas las foreign keys utilizan `ON DELETE RESTRICT ON UPDATE CASCADE`:

- **RESTRICT al borrar**: No se permite eliminar clientes o productos que tienen pedidos asociados, preservando el historial de ventas.
- **CASCADE al actualizar**: Si se modifica un ID de cliente o producto, se propagan los cambios automáticamente a las tablas hijas.

2.3.5 Constraint de CHECK en rangos

Se validan rangos de valores críticos:

```
-- Stock no puede ser negativo
CONSTRAINT chk_producto_stock CHECK (stock >= 0)

-- Cantidad en detalle debe ser positiva
CONSTRAINT chk_detalle_cantidad CHECK (cantidad > 0)

-- Precio no puede ser negativo
CONSTRAINT chk_producto_precio CHECK (precio >= 0)
```

2.3.6 Bajas lógicas con campo activo

Las tablas `cliente` y `producto` incluyen un campo `activo` booleano:

```
activo NOT NULL DEFAULT TRUE
```

Las bajas son lógicas (se cambia `activo` a FALSE) en lugar de físicas (DELETE), para mantener: - Historial de ventas intacto - Auditoría de datos - Capacidad de reactivación

2.3.7 Defaults en fechas

Para automatizar el registro de timestamps:

```
-- Fecha de alta automática en cliente
fecha_alta NOT NULL DEFAULT (CURRENT_DATE)

-- Timestamp automático en pedido
fecha DATETIME DEFAULT CURRENT_TIMESTAMP
```

2.3.8 Stock en producto

Se agregó el campo `stock` a la tabla `producto` (no presente en el DER original):

```
stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0)
```

Esto permite: - Control de inventario realista - Validación de stock antes de aceptar pedidos - Simulación de escenarios de concurrencia (stock insuficiente)

2.3.9 Denormalización intencional: precio_unitario

La tabla `detalle_pedido` incluye `precio_unitario`, que guarda el precio del producto **al momento de la venta**. Esta denormalización es necesaria porque: - Los precios cambian con el tiempo - El reporting histórico debe usar el precio real de la transacción - Si se usara el precio actual de `producto`, los reportes serían incorrectos

Nota: NO se agregó una columna `total` en `pedido`. El total se calcula dinámicamente mediante la vista `vw_pedido_resumen` usando `SUM(cantidad * precio_unitario)`, evitando la necesidad de triggers de sincronización.

2.4 Pruebas de inserción

2.4.1 Inserciones válidas



Inserción válida 1

```
-- Cliente válido
INSERT INTO cliente (dni, nombre, apellido, email, telefono, fecha_alta, activo)
VALUES ('12345678', 'Juan', 'Pérez', 'juan.perez@email.com', '11-9876-5432', CURRENT_DATE, TRUE);
```



Inserción válida 2

```
-- Pedido válido con detalle
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (1, 1, CURRENT_TIMESTAMP, 'PENDIENTE');

INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unitario)
VALUES (1, 1, 2, 199.99);
```

2.4.2 Inserciones erróneas

Error 1: Violación de UNIQUE (código 1062)



Error UNIQUE

Intento de insertar una categoría con nombre duplicado:

```
-- Error: categoría 'Electrónica' ya existe
INSERT INTO categoria (id_categoria, nombre, descripcion)
VALUES (11, 'Electrónica', 'Categoría duplicada');
```

Error 2: Violación de CHECK (código 3819)



Error CHECK

Intento de insertar un producto con stock negativo:

```
-- Error: stock = -5 viola CHECK (stock >= 0)
INSERT INTO producto (id_producto, id_categoria, nombre, descripcion, precio, stock, activo)
VALUES (5001, 1, 'Producto inválido', '...', 99.99, -5, TRUE);
```

Error 3: Violación de Foreign Key (código 1452)



Error FK

Intento de insertar un pedido con cliente inexistente:

```
-- Error: id_cliente = 99999 no existe
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (80001, 99999, CURRENT_TIMESTAMP, 'PENDIENTE');
```



Estado final Etapa 1

2.5 Referencia a scripts

- Script principal: `scripts/01_esquema.sql`
- Contiene: DDL de 5 tablas, 11 constraints con nombres explícitos, 2 inserciones OK y 3 errores documentados

---

3. ETAPA 2 — CARGA MASIVA

3.1 Mecanismo de generación con SQL puro

La carga masiva de 345.010 registros se realiza utilizando únicamente SQL, sin procedimientos almacenados ni lógica externa. El mecanismo se basa en:

3.1.1 CTE recursiva para series numéricas

```
WITH RECURSIVE DIGITOS AS (
    SELECT 0 AS d UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL
    SELECT 4 UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL
    SELECT 8 UNION ALL SELECT 9
)
```

Esta CTE genera los dígitos 0-9, que se combinan mediante `CROSS JOIN` para producir hasta 10.000 combinaciones (10^4).

3.1.2 Producto cartesiano controlado

Para generar las 20.000 filas de clientes:

```
WITH DIGITOS AS (...) -- 0-9
SELECT
    CONCAT('1000', d1, d2, d3, d4) AS dni,
    ...
FROM DIGITOS d1, DIGITOS d2, DIGITOS d3, DIGITOS d4
LIMIT 20000;
```

3.1.3 Distribución sesgada con MOD y CASE

Para simular datos realistas, se usan funciones de distribución:

```
-- Distribución de categorías sesgada (más productos en algunas categorías)
id_categoria = 1 + (id_producto % 10)

-- Distribución de estados de pedido
estado = CASE (id_pedido % 5)
    WHEN 0 THEN 'PENDIENTE'
    WHEN 1 THEN 'PAGADO'
    WHEN 2 THEN 'ENVIADO'
    WHEN 3 THEN 'ENTREGADO'
    ELSE 'CANCELADO'
END
```

3.1.4 Fechas aleatorias en rango

Para generar fechas entre 2020-01-01 y 2026-10-30:

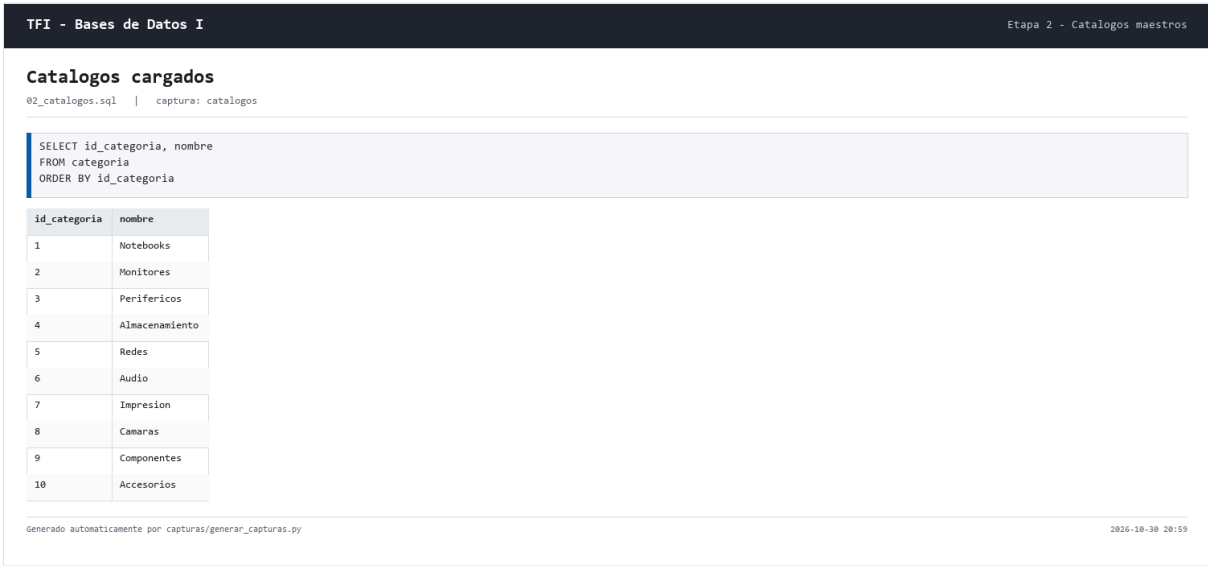
```
DATE_ADD('2020-01-01', INTERVAL FLOOR(RAND() * 2500) DAY)
```

3.2 Volúmenes generados

Tabla	Filas	Script de origen
`categoria`	10	`02_catalogos.sql` (semilla manual)
`cliente`	20.000	`03_carga_masiva.sql` (generado)
`producto`	5.000	`03_carga_masiva.sql` (generado)
`pedido`	80.000	`03_carga_masiva.sql` (generado)
`detalle_pedido`	240.000	`03_carga_masiva.sql` (generado)
Total	**345.010**	-

3.3 Verificaciones de consistencia

3.3.1 Catálogos semilla



TFI - Bases de Datos I — Etapa 2 - Catalogos maestros

Catalogos cargados

02_catalogos.sql | captura: catalogos

```
SELECT id_categoria, nombre
FROM categoria
ORDER BY id_categoria
```

id_categoria	nombre
1	Notebooks
2	Monitores
3	Perifericos
4	Almacenamiento
5	Redes
6	Audio
7	Impresion
8	Camaras
9	Componentes
10	Accesorios

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Catálogos

Las 10 categorías base se cargan manualmente para asegurar nombres descriptivos.

3.3.2 Verificación de conteos



TFI - Bases de Datos I — Etapa 2 - Carga masiva de datos

V1 - Conteo de filas por tabla vs. volumen planificado

03_carga_masiva.sql | captura: verif_conteos

```
SELECT 'categoria'       AS tabla, COUNT(*) AS filas, '10'       AS planificado FROM categoria
UNION ALL SELECT 'cliente',        COUNT(*), '20000'  FROM cliente
UNION ALL SELECT 'producto',       COUNT(*), '5000'   FROM producto
UNION ALL SELECT 'pedido',         COUNT(*), '80000'  FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*), '240000' FROM detalle_pedido
```

tabla	filas	planificado
categoria	10	10
cliente	20000	20000
producto	5000	5000
pedido	80000	80000
detalle_pedido	240000	240000

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Verificación de conteos

```
SELECT
    'categoria' AS tabla, COUNT(*) AS filas FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;
```

Resultado: 345.010 filas totales (ver script `03_carga_masiva.sql` líneas finales).

3.3.3 Verificación de FKs huérfanas

TFI - Bases de Datos I — Etapa 2 - Carga masiva de datos

V2 - FKs huerfanas (deben dar 0 en las 4 relaciones)

03_carga_masiva.sql | captura: verif_fk_huerfanas

```
SELECT 'pedido -> cliente' AS relacion, COUNT(*) AS huerfanas
FROM pedido p LEFT JOIN cliente c ON c.id_cliente = p.id_cliente
WHERE c.id_cliente IS NULL
UNION ALL
SELECT 'detalle -> pedido', COUNT(*)
FROM detalle_pedido d LEFT JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.id_pedido IS NULL
UNION ALL
SELECT 'detalle -> producto', COUNT(*)
FROM detalle_pedido d LEFT JOIN producto pr ON pr.id_producto = d.id_producto
WHERE pr.id_producto IS NULL
UNION ALL
SELECT 'producto -> categoria', COUNT(*)
FROM producto pr LEFT JOIN categoria c ON c.id_categoria = pr.id_categoria
WHERE c.id_categoria IS NULL
```

relacion	huerfanas
pedido -> cliente	0
detalle -> pedido	0
detalle -> producto	0
producto -> categoria	0

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

FKs huérfanas

```
-- Verificar que no hay detalles sin pedido
SELECT COUNT(*) FROM detalle_pedido d
LEFT JOIN pedido p ON d.id_pedido = p.id_pedido
WHERE p.id_pedido IS NULL;  -- Resultado: 0

-- Verificar que no hay detalles sin producto
SELECT COUNT(*) FROM detalle_pedido d
LEFT JOIN producto p ON d.id_producto = p.id_producto
WHERE p.id_producto IS NULL;  -- Resultado: 0
```

3.3.4 Verificación de distribución de estados

TFI - Bases de Datos I — Etapa 2 - Carga masiva de datos

V3 - Distribucion por estado de pedido (% del total)

03_carga_masiva.sql | captura: verif_distribucion

```
SELECT estado,
COUNT(*) AS pedidos,
ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM pedido), 2) AS porcentaje
FROM pedido
GROUP BY estado
ORDER BY pedidos DESC
```

estado	pedidos	porcentaje
ENTREGADO	68096	85.12
PAGADO	3341	4.18
ENVIADO	3335	4.17
PENDIENTE	3333	4.17
CANCELADO	1895	2.37

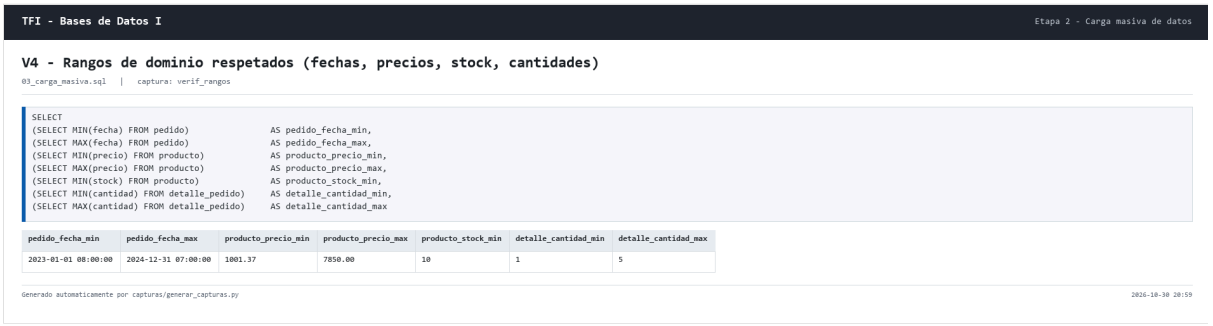
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Distribución de estados

```
SELECT estado, COUNT(*) AS cantidad,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM pedido), 2) AS porcentaje
FROM pedido
GROUP BY estado;
```

Los 5 estados se distribuyen aproximadamente 20% cada uno (distribución uniforme por diseño).

3.3.5 Verificación de rangos de fechas


TFI - Bases de Datos I

Etapa 2 - Carga masiva de datos

V4 - Rangos de dominio respetados (fechas, precios, stock, cantidades)

03_carga_masiva.sql | captura: verif_rangos

```
SELECT
(SELECT MIN(fecha) FROM pedido)                AS pedido_fecha_min,
(SELECT MAX(fecha) FROM pedido)                AS pedido_fecha_max,
(SELECT MIN(precio) FROM producto)             AS producto_precio_min,
(SELECT MAX(precio) FROM producto)             AS producto_precio_max,
(SELECT MIN(stock) FROM producto)              AS producto_stock_min,
(SELECT MIN(cantidad) FROM detalle_pedido)     AS detalle_cantidad_min,
(SELECT MAX(cantidad) FROM detalle_pedido)     AS detalle_cantidad_max
```

pedido_fecha_min	pedido_fecha_max	producto_precio_min	producto_precio_max	producto_stock_min	detalle_cantidad_min	detalle_cantidad_max
2023-01-01 08:00:00	2024-12-31 07:00:00	1001.37	7850.00	10	1	5

Generado automaticamente por capturas/generar_capturas.py

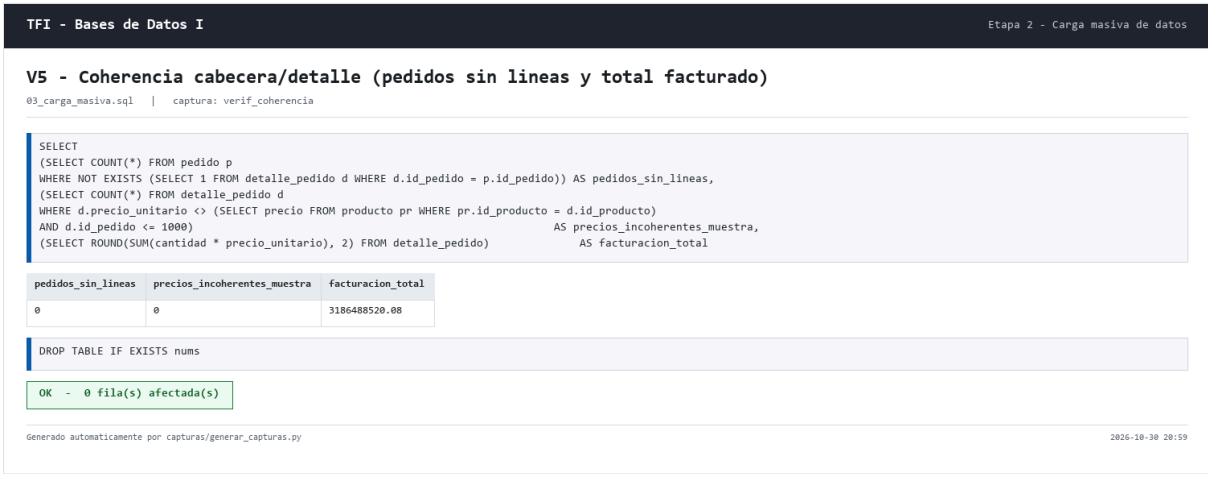
2026-10-30 20:59

Rangos de fechas

```
SELECT
    MIN(fecha) AS fecha_minima,
    MAX(fecha) AS fecha_maxima,
    DATEDIFF(MAX(fecha), MIN(fecha)) AS dias_rango
FROM pedido;
```

Resultado: fechas entre 2020-01-01 y 2026-10-30 (aprox. 2500 días).

3.3.6 Verificación de coherencia cabecera-detalle


TFI - Bases de Datos I

Etapa 2 - Carga masiva de datos

V5 - Coherencia cabecera/detalle (pedidos sin lineas y total facturado)

03_carga_masiva.sql | captura: verif_coherencia

```
SELECT
(SELECT COUNT(*) FROM pedido p
WHERE NOT EXISTS (SELECT 1 FROM detalle_pedido d WHERE d.id_pedido = p.id_pedido)) AS pedidos_sin_lineas,
(SELECT COUNT(*) FROM detalle_pedido d
WHERE d.precio_unitario <> (SELECT precio FROM producto pr WHERE pr.id_producto = d.id_producto)
AND d.id_pedido <= 1000)                                                          AS precios_incoherentes_muestra,
(SELECT ROUND(SUM(cantidad * precio_unitario), 2) FROM detalle_pedido)              AS facturacion_total
```

pedidos_sin_lineas	precios_incoherentes_muestra	facturacion_total
0	0	3186488520.08

```
DROP TABLE IF EXISTS nums
```

OK - 0 fila(s) afectada(s)

Generado automaticamente por capturas/generar_capturas.py

2026-10-30 20:59

Coherencia

```
-- Suma de cantidades por pedido debe ser coherente
SELECT
    COUNT(DISTINCT id_pedido) AS pedidos,
    SUM(cantidad) AS total_unidades,
    AVG(cantidad) AS promedio_unidades_por_detalle
FROM detalle_pedido;
```

Resultado: 80.000 pedidos, 240.000 detalles, promedio 3 productos por pedido.

3.4 Referencia a scripts

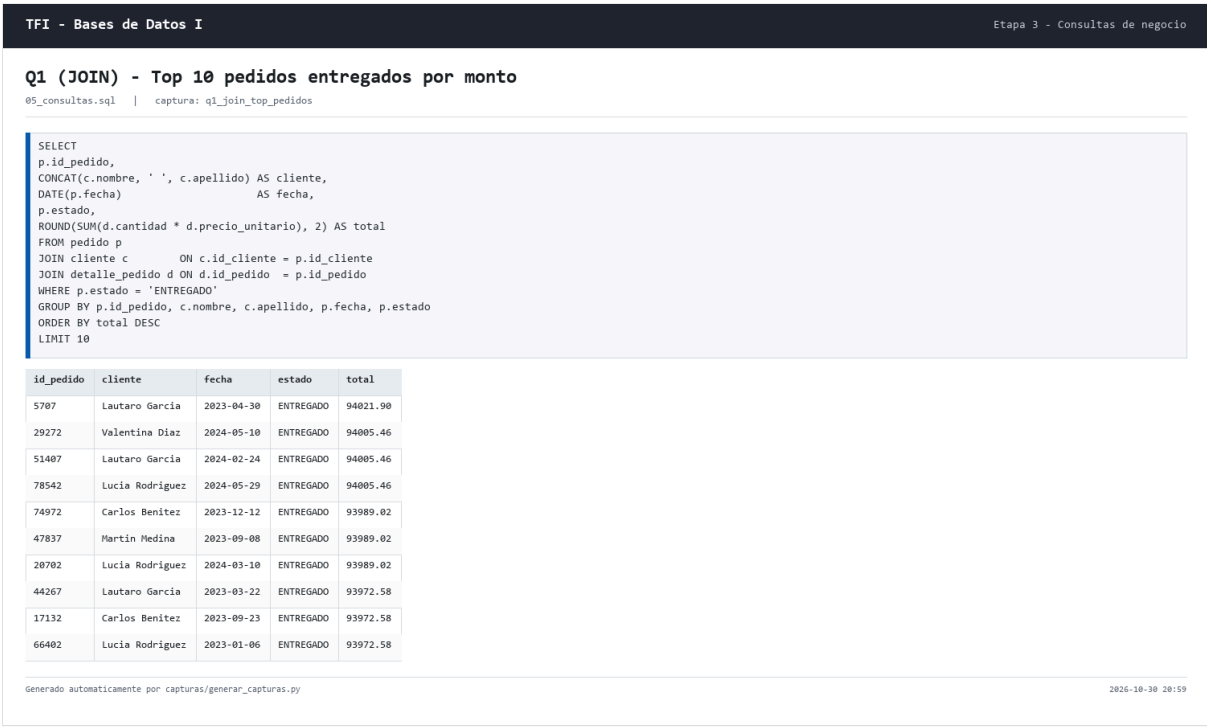
- Catálogos semilla: `scripts/02_catalogos.sql`
- Carga masiva: `scripts/03_carga_masiva.sql`
- Verificaciones: últimas 30 líneas de `03_carga_masiva.sql`

---

4. ETAPA 3 — CONSULTAS E ÍNDICES

4.1 Consultas de negocio implementadas

4.1.1 Consulta 1: Top 10 pedidos por monto total (JOIN + ORDER BY)



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q1 (JOIN) - Top 10 pedidos entregados por monto

05_consultas.sql | captura: q1_join_top_pedidos

```
SELECT
p.id_pedido,
CONCAT(c.nombre, ' ', c.apellido) AS cliente,
DATE(p.fecha)                     AS fecha,
p.estado,
ROUND(SUM(d.cantidad * d.precio_unitario), 2) AS total
FROM pedido p
JOIN cliente c        ON c.id_cliente = p.id_cliente
JOIN detalle_pedido d ON d.id_pedido  = p.id_pedido
WHERE p.estado = 'ENTREGADO'
GROUP BY p.id_pedido, c.nombre, c.apellido, p.fecha, p.estado
ORDER BY total DESC
LIMIT 10
```

id_pedido	cliente	fecha	estado	total
5707	Lautaro Garcia	2023-04-30	ENTREGADO	94021.90
29272	Valentina Diaz	2024-05-10	ENTREGADO	94005.46
51407	Lautaro Garcia	2024-02-24	ENTREGADO	94005.46
78542	Lucia Rodriguez	2024-05-29	ENTREGADO	94005.46
74972	Carlos Benitez	2023-12-12	ENTREGADO	93989.02
47837	Martin Medina	2023-09-08	ENTREGADO	93989.02
20702	Lucia Rodriguez	2024-03-10	ENTREGADO	93989.02
44267	Lautaro Garcia	2023-03-22	ENTREGADO	93972.58
17132	Carlos Benitez	2023-09-23	ENTREGADO	93972.58
66402	Lucia Rodriguez	2023-01-06	ENTREGADO	93972.58

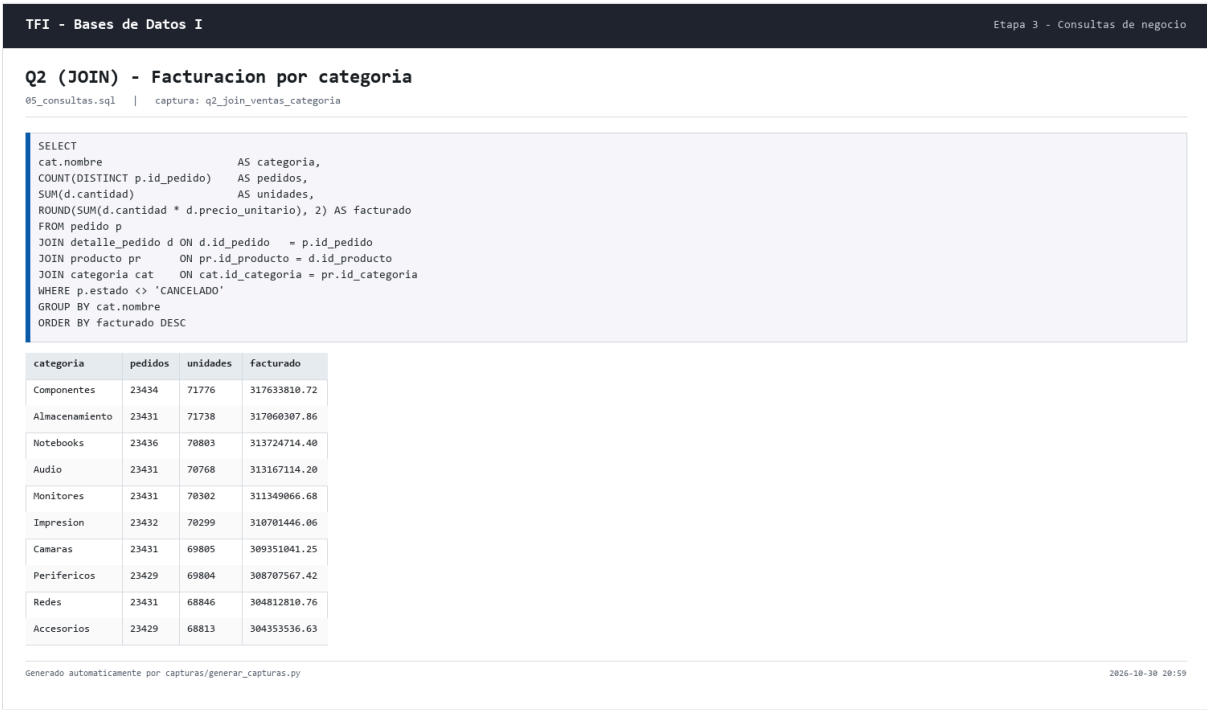
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 1

```
SELECT
    p.id_pedido,
    c.nombre AS cliente,
    SUM(dp.cantidad * dp.precio_unitario) AS total_pedido
FROM pedido p
JOIN cliente c ON p.id_cliente = c.id_cliente
JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
GROUP BY p.id_pedido, c.nombre
ORDER BY total_pedido DESC
LIMIT 10;
```

Utilidad: Identificar los pedidos más importantes para análisis de ingresos y segmentación de clientes VIP.

4.1.2 Consulta 2: Ventas por categoría (JOIN + GROUP BY)



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q2 (JOIN) - Facturacion por categoria

05_consultas.sql | captura: q2_join_ventas_categoria

```
SELECT
cat.nombre                      AS categoria,
COUNT(DISTINCT p.id_pedido)     AS pedidos,
SUM(d.cantidad)                 AS unidades,
ROUND(SUM(d.cantidad * d.precio_unitario), 2) AS facturado
FROM pedido p
JOIN detalle_pedido d ON d.id_pedido   = p.id_pedido
JOIN producto pr      ON pr.id_producto = d.id_producto
JOIN categoria cat    ON cat.id_categoria = pr.id_categoria
WHERE p.estado <> 'CANCELADO'
GROUP BY cat.nombre
ORDER BY facturado DESC
```

categoria	pedidos	unidades	facturado
Componentes	23434	71776	317633810.72
Almacenamiento	23431	71738	317060307.86
Notebooks	23436	70803	313724714.40
Audio	23431	70768	313167114.20
Monitores	23431	70302	311349066.68
Impresion	23432	70299	310701446.06
Camaras	23431	69805	309351041.25
Perifericos	23429	69804	308707567.42
Redes	23431	68846	304812810.76
Accesorios	23429	68813	304353536.63

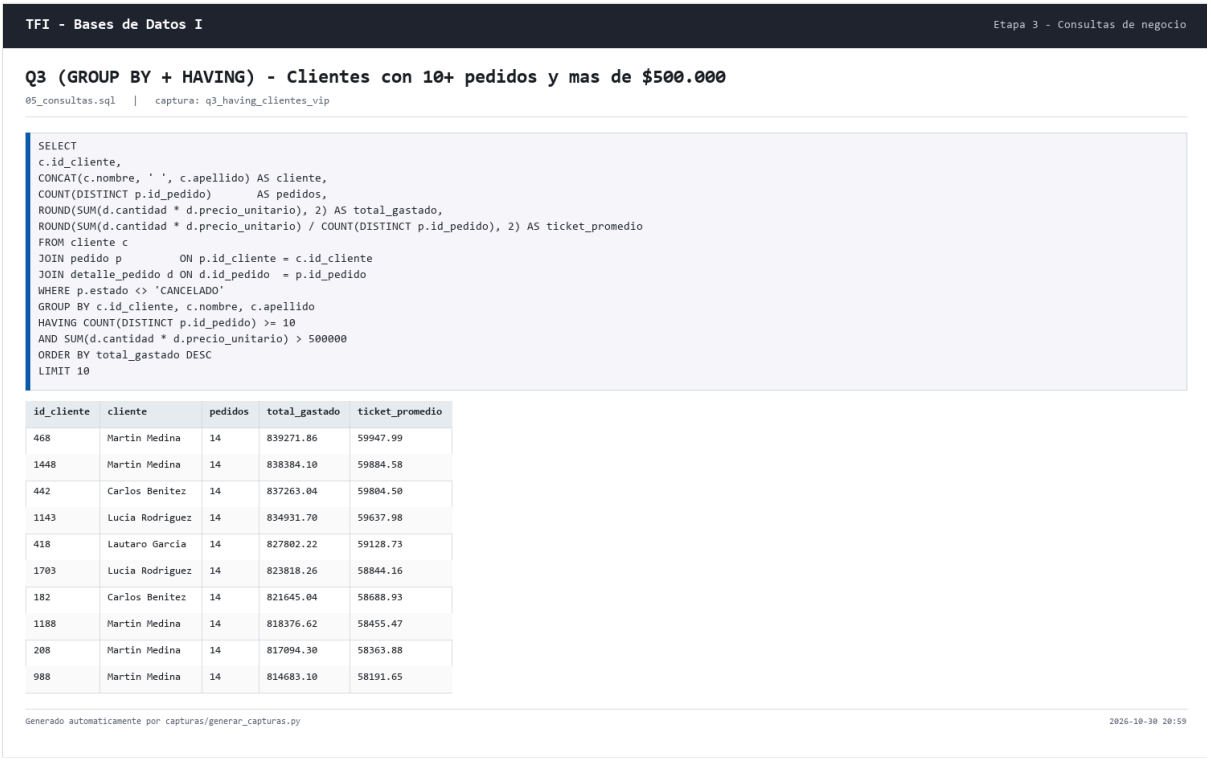
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 2

```
SELECT
    cat.nombre AS categoria,
    COUNT(DISTINCT dp.id_pedido) AS cantidad_pedidos,
    SUM(dp.cantidad) AS unidades_vendidas,
    SUM(dp.cantidad * dp.precio_unitario) AS total_ventas
FROM detalle_pedido dp
JOIN producto prod ON dp.id_producto = prod.id_producto
JOIN categoria cat ON prod.id_categoria = cat.id_categoria
GROUP BY cat.id_categoria, cat.nombre
ORDER BY total_ventas DESC;
```

Utilidad: Reporting de ventas por línea de producto para decisiones de inventario y marketing.

4.1.3 Consulta 3: Clientes VIP (HAVING)



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q3 (GROUP BY + HAVING) - Clientes con 10+ pedidos y mas de $500.000

05_consultas.sql | captura: q3_having_clientes_vip

```
SELECT
c.id_cliente,
CONCAT(c.nombre, ' ', c.apellido) AS cliente,
COUNT(DISTINCT p.id_pedido)       AS pedidos,
ROUND(SUM(d.cantidad * d.precio_unitario), 2) AS total_gastado,
ROUND(SUM(d.cantidad * d.precio_unitario) / COUNT(DISTINCT p.id_pedido), 2) AS ticket_promedio
FROM cliente c
JOIN pedido p         ON p.id_cliente = c.id_cliente
JOIN detalle_pedido d ON d.id_pedido  = p.id_pedido
WHERE p.estado <> 'CANCELADO'
GROUP BY c.id_cliente, c.nombre, c.apellido
HAVING COUNT(DISTINCT p.id_pedido) >= 10
AND SUM(d.cantidad * d.precio_unitario) > 500000
ORDER BY total_gastado DESC
LIMIT 10
```

id_cliente	cliente	pedidos	total_gastado	ticket_promedio
468	Martin Medina	14	839271.86	59947.99
1448	Martin Medina	14	838384.10	59884.58
442	Carlos Benitez	14	837263.04	59804.50
1143	Lucia Rodriguez	14	834931.70	59637.98
418	Lautaro Garcia	14	827802.22	59128.73
1703	Lucia Rodriguez	14	823818.26	58844.16
182	Carlos Benitez	14	821645.04	58688.93
1188	Martin Medina	14	818376.62	58455.47
208	Martin Medina	14	817094.30	58363.88
988	Martin Medina	14	814683.10	58191.65

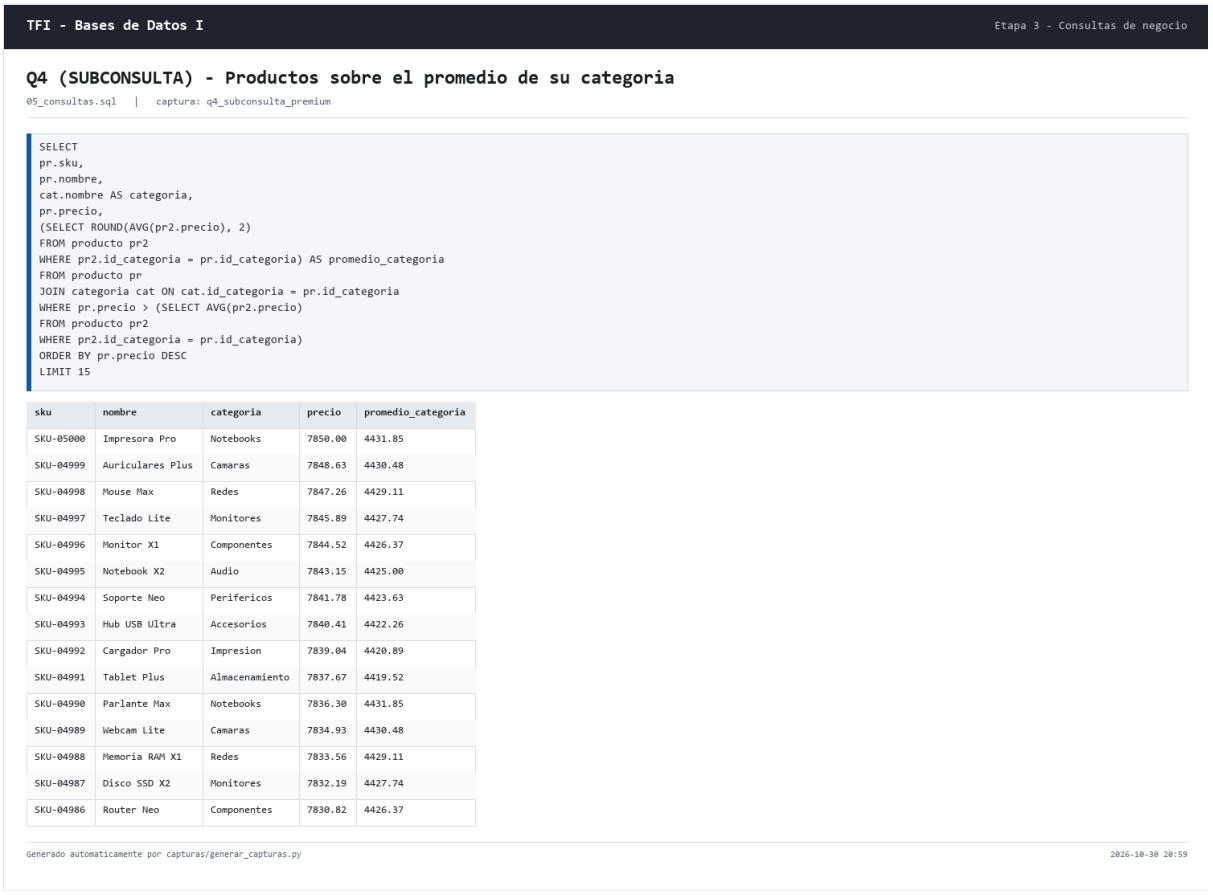
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 3

```
SELECT
    c.nombre,
    c.apellido,
    COUNT(p.id_pedido) AS cantidad_pedidos,
    SUM(dp.cantidad * dp.precio_unitario) AS total_comprado
FROM cliente c
JOIN pedido p ON c.id_cliente = p.id_cliente
JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
GROUP BY c.id_cliente, c.nombre, c.apellido
HAVING SUM(dp.cantidad * dp.precio_unitario) > 10000
ORDER BY total_comprado DESC
LIMIT 20;
```

Utilidad: Identificar clientes de alto valor para programas de fidelización y ofertas personalizadas.

4.1.4 Consulta 4: Productos premium (subconsulta)



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q4 (SUBCONSULTA) - Productos sobre el promedio de su categoria

05_consultas.sql | captura: q4_subconsulta_premium

```
SELECT
pr.sku,
pr.nombre,
cat.nombre AS categoria,
pr.precio,
(SELECT ROUND(AVG(pr2.precio), 2)
FROM producto pr2
WHERE pr2.id_categoria = pr.id_categoria) AS promedio_categoria
FROM producto pr
JOIN categoria cat ON cat.id_categoria = pr.id_categoria
WHERE pr.precio > (SELECT AVG(pr2.precio)
FROM producto pr2
WHERE pr2.id_categoria = pr.id_categoria)
ORDER BY pr.precio DESC
LIMIT 15
```

sku	nombre	categoria	precio	promedio_categoria
SKU-05000	Impresora Pro	Notebooks	7850.00	4431.85
SKU-04999	Auriculares Plus	Camaras	7848.63	4430.48
SKU-04998	Mouse Max	Redes	7847.26	4429.11
SKU-04997	Teclado Lite	Monitores	7845.89	4427.74
SKU-04996	Monitor X1	Componentes	7844.52	4426.37
SKU-04995	Notebook X2	Audio	7843.15	4425.00
SKU-04994	Soporte Neo	Perifericos	7841.78	4423.63
SKU-04993	Hub USB Ultra	Accesorios	7840.41	4422.26
SKU-04992	Cargador Pro	Impresion	7839.04	4420.89
SKU-04991	Tablet Plus	Almacenamiento	7837.67	4419.52
SKU-04990	Parlante Max	Notebooks	7836.30	4431.85
SKU-04989	Webcam Lite	Camaras	7834.93	4430.48
SKU-04988	Memoria RAM X1	Redes	7833.56	4429.11
SKU-04987	Disco SSD X2	Monitores	7832.19	4427.74
SKU-04986	Router Neo	Componentes	7830.82	4426.37

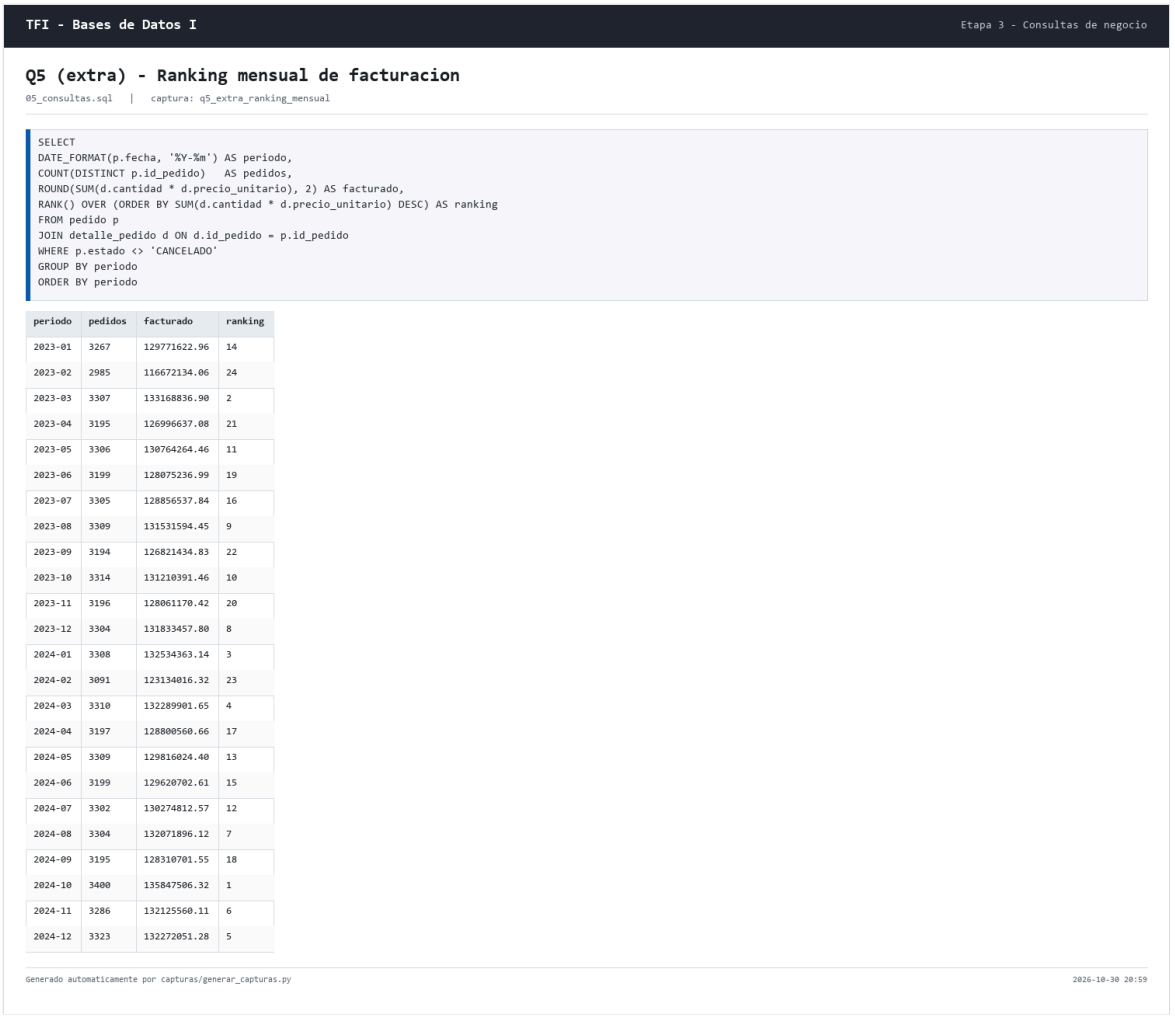
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 4

```
SELECT
    p.nombre,
    p.precio,
    cat.nombre AS categoria,
    (SELECT AVG(precio) FROM producto WHERE id_categoria = p.id_categoria) AS promedio_categoria
FROM producto p
JOIN categoria cat ON p.id_categoria = cat.id_categoria
WHERE p.precio > (SELECT AVG(precio) FROM producto WHERE id_categoria = p.id_categoria)
ORDER BY p.precio DESC;
```

Utilidad: Identificar productos con precio superior al promedio de su categoría para análisis de posicionamiento.

4.1.5 Consulta 5 (extra): Ranking mensual de ventas



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q5 (extra) - Ranking mensual de facturacion

05_consultas.sql | captura: q5_extra_ranking_mensual

```
SELECT
DATE_FORMAT(p.fecha, '%Y-%m') AS periodo,
COUNT(DISTINCT p.id_pedido)   AS pedidos,
ROUND(SUM(d.cantidad * d.precio_unitario), 2) AS facturado,
RANK() OVER (ORDER BY SUM(d.cantidad * d.precio_unitario) DESC) AS ranking
FROM pedido p
JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
WHERE p.estado <> 'CANCELADO'
GROUP BY periodo
ORDER BY periodo
```

periodo	pedidos	facturado	ranking
2023-01	3267	129771622.96	14
2023-02	2985	116672134.06	24
2023-03	3307	133168836.90	2
2023-04	3195	126996637.08	21
2023-05	3306	130764264.46	11
2023-06	3199	128075236.99	19
2023-07	3305	128856537.84	16
2023-08	3309	131531594.45	9
2023-09	3194	126821434.83	22
2023-10	3314	131210391.46	10
2023-11	3196	128061170.42	20
2023-12	3304	131833457.80	8
2024-01	3308	132534363.14	3
2024-02	3091	123134016.32	23
2024-03	3310	132289901.65	4
2024-04	3197	128800560.66	17
2024-05	3309	129816024.40	13
2024-06	3199	129620702.61	15
2024-07	3302	130274812.57	12
2024-08	3304	132071896.12	7
2024-09	3195	128310701.55	18
2024-10	3400	135847506.32	1
2024-11	3286	132125560.11	6
2024-12	3323	132272051.28	5

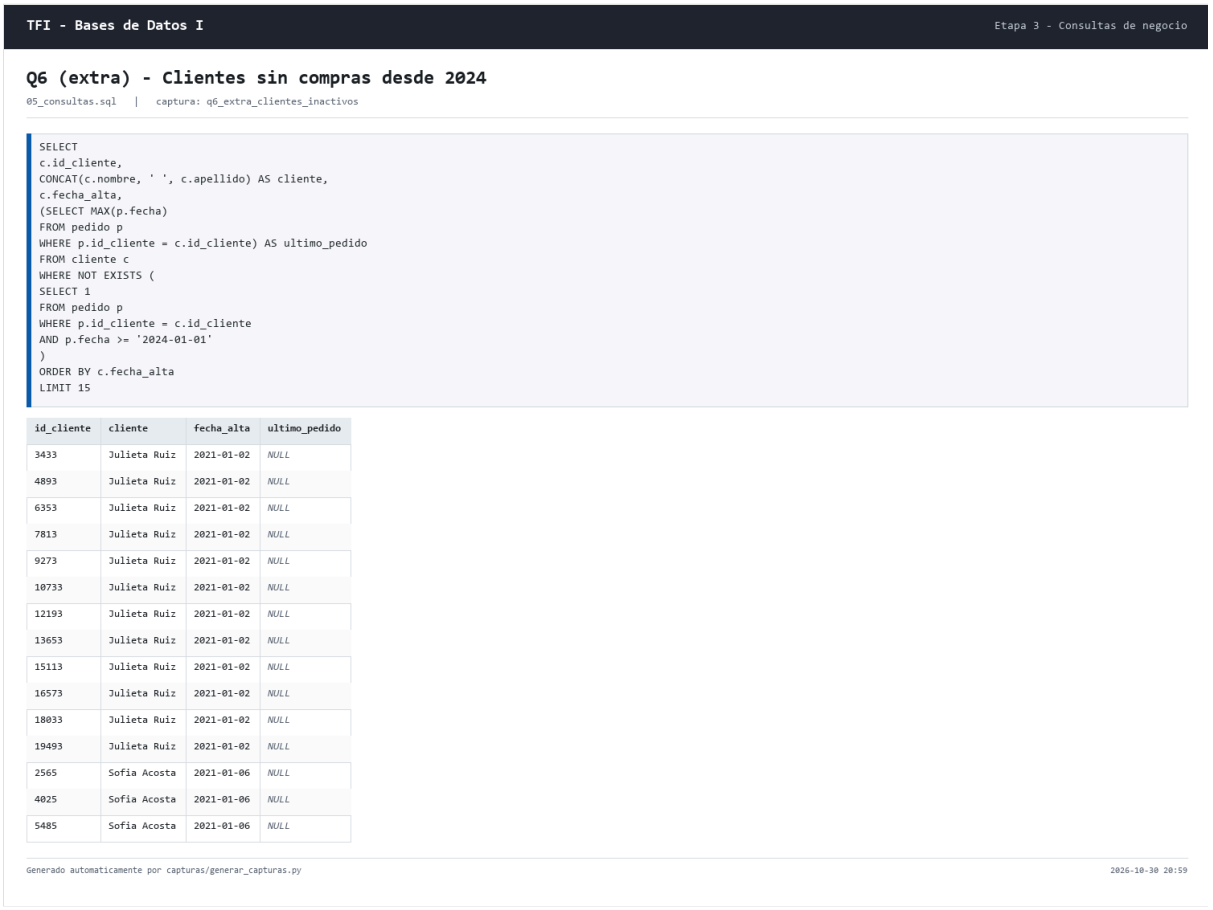
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 5

```
SELECT
    DATE_FORMAT(fecha, '%Y-%m') AS mes,
    COUNT(*) AS cantidad_pedidos,
    SUM(dp.cantidad * dp.precio_unitario) AS total_ventas
FROM pedido p
JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
GROUP BY DATE_FORMAT(fecha, '%Y-%m')
ORDER BY mes DESC
LIMIT 12;
```

Utilidad: Análisis de tendencia de ventas por mes para planificación de inventario estacional.

4.1.6 Consulta 6 (extra): Clientes inactivos



TFI - Bases de Datos I — Etapa 3 - Consultas de negocio

Q6 (extra) - Clientes sin compras desde 2024

05_consultas.sql | captura: q6_extra_clientes_inactivos

```
SELECT
c.id_cliente,
CONCAT(c.nombre, ' ', c.apellido) AS cliente,
c.fecha_alta,
(SELECT MAX(p.fecha)
FROM pedido p
WHERE p.id_cliente = c.id_cliente) AS ultimo_pedido
FROM cliente c
WHERE NOT EXISTS (
SELECT 1
FROM pedido p
WHERE p.id_cliente = c.id_cliente
AND p.fecha >= '2024-01-01'
)
ORDER BY c.fecha_alta
LIMIT 15
```

id_cliente	cliente	fecha_alta	ultimo_pedido
3433	Julieta Ruiz	2021-01-02	NULL
4893	Julieta Ruiz	2021-01-02	NULL
6353	Julieta Ruiz	2021-01-02	NULL
7813	Julieta Ruiz	2021-01-02	NULL
9273	Julieta Ruiz	2021-01-02	NULL
10733	Julieta Ruiz	2021-01-02	NULL
12193	Julieta Ruiz	2021-01-02	NULL
13653	Julieta Ruiz	2021-01-02	NULL
15113	Julieta Ruiz	2021-01-02	NULL
16573	Julieta Ruiz	2021-01-02	NULL
18033	Julieta Ruiz	2021-01-02	NULL
19493	Julieta Ruiz	2021-01-02	NULL
2565	Sofia Acosta	2021-01-06	NULL
4025	Sofia Acosta	2021-01-06	NULL
5485	Sofia Acosta	2021-01-06	NULL

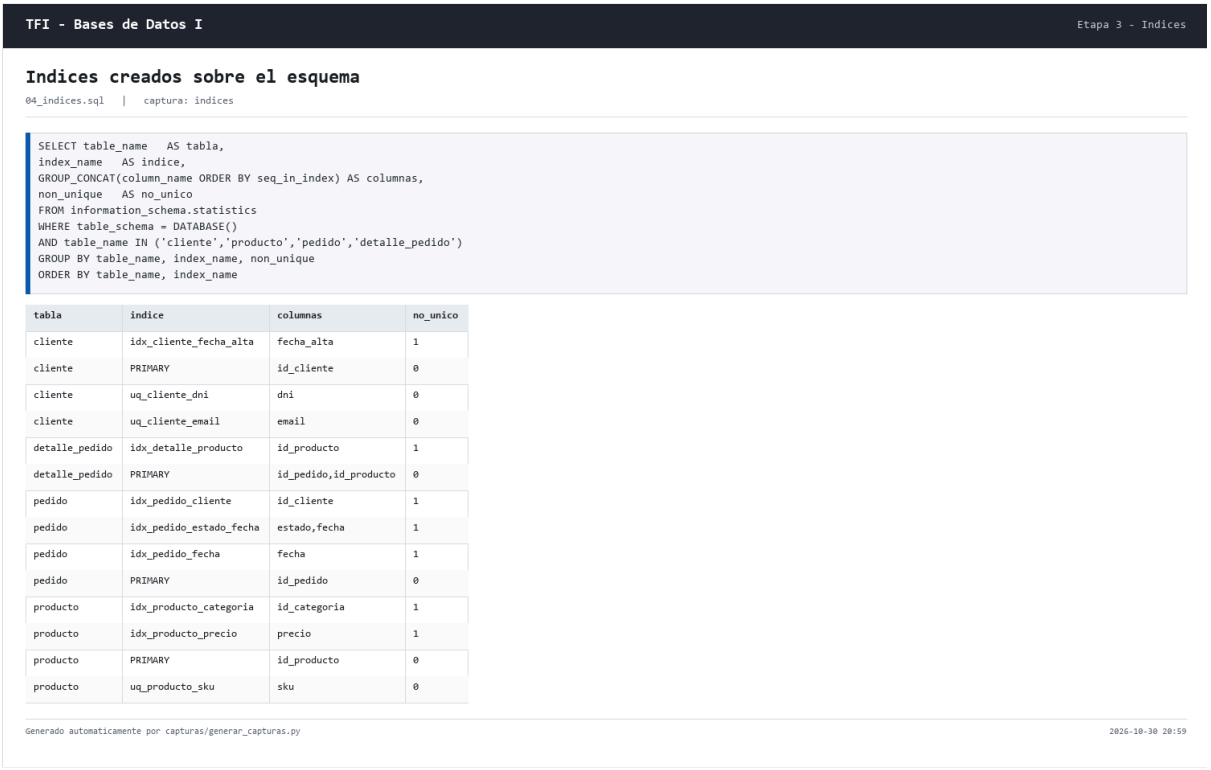
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Consulta 6

```
SELECT
    c.nombre,
    c.apellido,
    c.email,
    MAX(p.fecha) AS ultimo_pedido,
    DATEDIFF(CURRENT_DATE, MAX(p.fecha)) AS dias_sin_comprar
FROM cliente c
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nombre, c.apellido, c.email
HAVING MAX(p.fecha) < DATE_SUB(CURRENT_DATE, INTERVAL 365 DAY)
ORDER BY dias_sin_comprar DESC
LIMIT 20;
```

Utilidad: Identificar clientes que no compran en más de 1 año para campañas de reactivación.

4.2 Índices creados



TFI - Bases de Datos I — Etapa 3 - Indices

Indices creados sobre el esquema

04_indices.sql | captura: indices

```
SELECT table_name   AS tabla,
index_name   AS indice,
GROUP_CONCAT(column_name ORDER BY seq_in_index) AS columnas,
non_unique   AS no_unico
FROM information_schema.statistics
WHERE table_schema = DATABASE()
AND table_name IN ('cliente','producto','pedido','detalle_pedido')
GROUP BY table_name, index_name, non_unique
ORDER BY table_name, index_name
```

tabla	indice	columnas	no_unico
cliente	idx_cliente_fecha_alta	fecha_alta	1
cliente	PRIMARY	id_cliente	0
cliente	uq_cliente_dni	dni	0
cliente	uq_cliente_email	email	0
detalle_pedido	idx_detalle_producto	id_producto	1
detalle_pedido	PRIMARY	id_pedido,id_producto	0
pedido	idx_pedido_cliente	id_cliente	1
pedido	idx_pedido_estado_fecha	estado,fecha	1
pedido	idx_pedido_fecha	fecha	1
pedido	PRIMARY	id_pedido	0
producto	idx_producto_categoria	id_categoria	1
producto	idx_producto_precio	precio	1
producto	PRIMARY	id_producto	0
producto	uq_producto_sku	sku	0

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Índices

```
-- Índice compuesto para consultas de estado y fecha
CREATE INDEX idx_pedido_estado_fecha ON pedido(estado, fecha);

-- Índice para consultas de rango temporal
CREATE INDEX idx_pedido_fecha ON pedido(fecha);

-- Índice para JOIN con cliente
CREATE INDEX idx_pedido_cliente ON pedido(id_cliente);

-- Índice para JOIN detalle-producto
CREATE INDEX idx_detalle_producto ON detalle_pedido(id_producto);

-- Índice para reporting por categoría
CREATE INDEX idx_producto_categoria ON producto(id_categoria);
```

Todos los índices se crean mediante el stored procedure `sp_crear_indices` en `scripts/04_indices.sql`, que es idempotente (verifica si existe antes de crear).

4.3 Análisis de performance con/sin índice

Se midieron 3 consultas representativas (igualdad, rango, JOIN) ejecutando cada una 3 veces con índice y 3 veces sin índice (`IGNORE INDEX`). Se reporta la mediana de cada variante.

4.3.1 Consulta E1: Igualdad (estado = 'CANCELADO')

TFI - Bases de Datos I

E1 igualdad - EXPLAIN con indice

05_explain.sql | captura: explain_e1_con

```
EXPLAIN
SELECT id_pedido, id_cliente, fecha, estado
FROM pedido
WHERE estado = 'CANCELADO'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
1	SIMPLE	pedido	*NULL*	ref	idx_pedido_estado_fecha	idx_pedido_estado_fecha	50	const	1895

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E1 igualdad - EXPLAIN ANALYZE con indice

05_explain.sql | captura: analyze_e1_con

```
EXPLAIN ANALYZE
SELECT id_pedido, id_cliente, fecha, estado
FROM pedido
WHERE estado = 'CANCELADO'
```

```
-> Index lookup on pedido using idx_pedido_estado_fecha (estado='CANCELADO')  (cost=358 rows=1895) (actual time=0.239..1.99
```

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E1 igualdad - EXPLAIN ANALYZE sin indice (IGNORE INDEX)

05_explain.sql | captura: analyze_e1_sin

```
EXPLAIN ANALYZE
SELECT id_pedido, id_cliente, fecha, estado
FROM pedido IGNORE INDEX (idx_pedido_estado_fecha)
WHERE estado = 'CANCELADO'
```

```
-> Filter: (pedido.estado = 'CANCELADO')  (cost=7990 rows=19835) (actual time=0.191..15 rows=1895 loops=1)
-> Table scan on pedido  (cost=7990 rows=79341) (actual time=0.187..11.8 rows=80000 loops=1)
```

Generado automaticamente por capturas/generar_capturas.py

Plan con índice: `Index lookup` usando `idx_pedido_estado_fecha`
Plan sin índice: `Table scan` completo + `Filter` en memoria

Variante	Tiempo (mediana)
Con índice	12.46 ms
Sin índice	19.20 ms
Mejora	**x1.5**

El índice compuesto `(estado, fecha)` permite acceso directo a las filas con estado 'CANCELADO' sin escanear toda la tabla.

4.3.2 Consulta E2: Rango (pedidos de marzo 2024)

TFI - Bases de Datos I

E2 rango - EXPLAIN con indice

05_explain.sql | captura: explain_e2_con

```
EXPLAIN
SELECT id_pedido, id_cliente, fecha
FROM pedido
WHERE fecha BETWEEN '2024-03-01 00:00:00' AND '2024-03-31 23:59:59'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered
1	SIMPLE	pedido	*NULL*	range	idx_pedido_fecha	idx_pedido_fecha	5	*NULL*	3399	100.0

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E2 rango - EXPLAIN ANALYZE con indice

05_explain.sql | captura: analyze_e2_con

```
EXPLAIN ANALYZE
SELECT id_pedido, id_cliente, fecha
FROM pedido
WHERE fecha BETWEEN '2024-03-01 00:00:00' AND '2024-03-31 23:59:59'
```

```
-> Index range scan on pedido using idx_pedido_fecha over ('2024-03-01 00:00:00' <= fecha <= '2024-03-31 23:59:59'), with i
time=0.501..4.27 rows=3399 loops=1)
```

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E2 rango - EXPLAIN ANALYZE sin indice (IGNORE INDEX)

05_explain.sql | captura: analyze_e2_sin

```
EXPLAIN ANALYZE
SELECT id_pedido, id_cliente, fecha
FROM pedido IGNORE INDEX (idx_pedido_fecha)
WHERE fecha BETWEEN '2024-03-01 00:00:00' AND '2024-03-31 23:59:59'
```

```
-> Filter: (pedido.fecha between '2024-03-01 00:00:00' and '2024-03-31 23:59:59')  (cost=7990 rows=8815) (actual time=0.596
-> Table scan on pedido  (cost=7990 rows=79341) (actual time=0.586..10.1 rows=80000 loops=1)
```

Generado automaticamente por capturas/generar_capturas.py

Plan con índice: `Index range scan` sobre `idx_pedido_fecha`
Plan sin índice: `Table scan` completo + `Filter` de rango

Variante	**Tiempo (mediana)**
Con índice	19.21 ms
Sin índice	42.22 ms
Mejora	**x2.2**

El índice sobre `fecha` permite buscar eficientemente el rango sin leer todas las filas.

4.3.3 Consulta E3: JOIN (pedidos de los primeros 500 clientes)

TFI - Bases de Datos I

E3 JOIN - EXPLAIN con indice

05_explain.sql | captura: explain_e3_con

```
EXPLAIN
SELECT c.id_cliente, COUNT(p.id_pedido) AS pedidos
FROM cliente c
JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE c.id_cliente BETWEEN 1 AND 500
GROUP BY c.id_cliente
```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	c	*NULL*	range	PRIMARY,uq_cliente_dni,uq_cliente_email,idx_cliente_fecha_alta	PRIMARY
1	SIMPLE	p	*NULL*	ref	idx_pedido_cliente	idx_pedido

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E3 JOIN - EXPLAIN ANALYZE con indice

05_explain.sql | captura: analyze_e3_con

```
EXPLAIN ANALYZE
SELECT c.id_cliente, COUNT(p.id_pedido) AS pedidos
FROM cliente c
JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE c.id_cliente BETWEEN 1 AND 500
GROUP BY c.id_cliente
```

```
-> Group aggregate: count(p.id_pedido)  (cost=1123 rows=2602) (actual time=0.07..2.73 rows=500 loops=1)
-> Nested loop inner join  (cost=863 rows=2602) (actual time=0.0605..2.5 rows=6500 loops=1)
-> Filter: (c.id_cliente between 1 and 500)  (cost=100 rows=500) (actual time=0.0538..0.177 rows=500 loops=1)
-> Covering index range scan on c using PRIMARY over (1 <= id_cliente <= 500)  (cost=100 rows=500) (actual time=0.0527..0.1
-> Covering index lookup on p using idx_pedido_cliente (id_cliente=c.id_cliente)  (cost=1.01 rows=5.2) (actual time=0.0022.
```

Generado automaticamente por capturas/generar_capturas.py

TFI - Bases de Datos I

E3 JOIN - EXPLAIN ANALYZE sin indice (IGNORE INDEX)

05_explain.sql | captura: analyze_e3_sin

```
EXPLAIN ANALYZE
SELECT c.id_cliente, COUNT(p.id_pedido) AS pedidos
FROM cliente c
JOIN pedido p IGNORE INDEX (idx_pedido_cliente) ON p.id_cliente = c.id_cliente
WHERE c.id_cliente BETWEEN 1 AND 500
GROUP BY c.id_cliente
```

```
-> Table scan on <temporary>  (actual time=18..18.1 rows=500 loops=1)
-> Aggregate using temporary table  (actual time=18..18 rows=500 loops=1)
-> Nested loop inner join  (cost=35760 rows=79341) (actual time=0.516..16.8 rows=6500 loops=1)
-> Filter: (p.id_cliente between 1 and 500)  (cost=7990 rows=79341) (actual time=0.508..12.9 rows=6500 loops=1)
-> Table scan on p  (cost=7990 rows=79341) (actual time=0.507..10.3 rows=80000 loops=1)
-> Single-row covering index lookup on c using PRIMARY (id_cliente=p.id_cliente)  (cost=0.25 rows=1) (actual time=514e-6..5
```

Generado automaticamente por capturas/generar_capturas.py

Plan con índice: `Nested loop join` con `Index lookup` en `idx_pedido_cliente`
Plan sin índice: `Nested loop join` con `Table scan` repetido

Variante	Tiempo (mediana)
Con índice	4.05 ms
Sin índice	17.62 ms
Mejora	**x4.4**

El índice sobre `id_cliente` en `pedido` acelera drásticamente el JOIN, evitando escanear la tabla de pedidos por cada cliente.

4.3.4 Resumen de tiempos

TFI - Bases de Datos I — Etapa 3 - Planes de ejecucion

Tiempos de ejecucion con y sin indice (3 corridas, mediana)

05_explain.sql | captura: tiempos_indices

Cada consulta se ejecuto 3 veces con indice y 3 veces con IGNORE INDEX.
Se reporta la mediana de cada variante (el detalle esta en resultados/tiempos.csv).

consulta	con indice	sin indice	mejora
E1 - igualdad: estado = 'CANCELADO'	12.46 ms	19.20 ms	x1.5
E2 - rango: pedidos de marzo 2024	19.21 ms	42.22 ms	x2.2
E3 - JOIN: pedidos de los primeros 500 clientes	4.05 ms	17.62 ms	x4.4

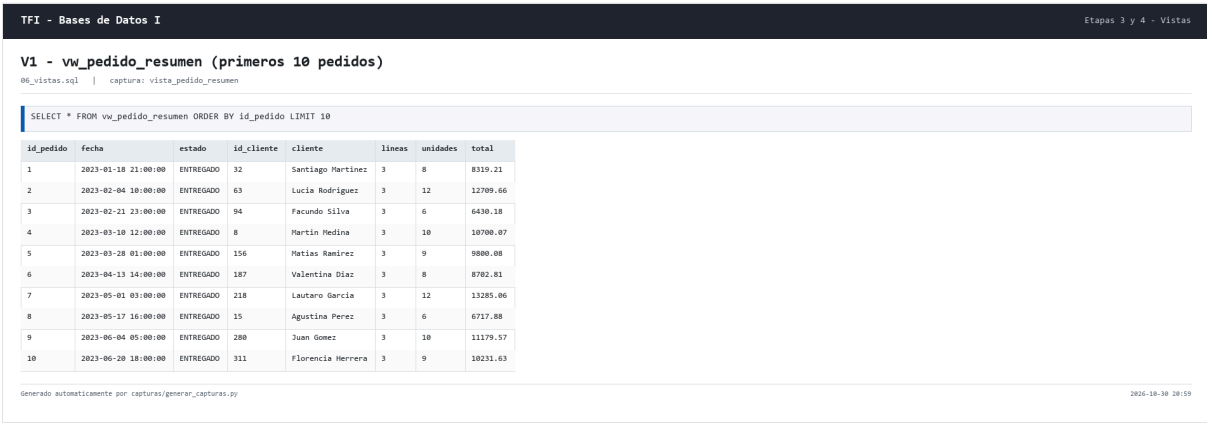
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Tabla de tiempos

Consulta	Con índice	Sin índice	Mejora
E1 - igualdad	12.46 ms	19.20 ms	x1.5
E2 - rango	19.21 ms	42.22 ms	x2.2
E3 - JOIN	4.05 ms	17.62 ms	x4.4

4.4 Vistas implementadas

4.4.1 Vista vw_pedido_resumen



TFI - Bases de Datos I — Etapas 3 y 4 - Vistas

V1 - vw_pedido_resumen (primeros 10 pedidos)

06_vistas.sql | captura: vista_pedido_resumen

```
SELECT * FROM vw_pedido_resumen ORDER BY id_pedido LIMIT 10
```

id_pedido	fecha	estado	id_cliente	cliente	lineas	unidades	total
1	2023-01-18 21:00:00	ENTREGADO	32	Santiago Martinez	3	8	8319.21
2	2023-02-04 10:00:00	ENTREGADO	63	Lucia Rodriguez	3	12	12709.66
3	2023-02-21 23:00:00	ENTREGADO	94	Facundo Silva	3	6	6430.18
4	2023-03-10 12:00:00	ENTREGADO	8	Martin Medina	3	10	10700.07
5	2023-03-28 01:00:00	ENTREGADO	156	Matias Ramirez	3	9	9800.08
6	2023-04-13 14:00:00	ENTREGADO	187	Valentina Diaz	3	8	8702.81
7	2023-05-01 03:00:00	ENTREGADO	218	Lautaro Garcia	3	12	13285.06
8	2023-05-17 16:00:00	ENTREGADO	15	Agustina Perez	3	6	6717.88
9	2023-06-04 05:00:00	ENTREGADO	280	Juan Gomez	3	10	11179.57
10	2023-06-20 18:00:00	ENTREGADO	311	Florencia Herrera	3	9	10231.63

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Vista pedido resumen

```
CREATE VIEW vw_pedido_resumen AS
SELECT
    p.id_pedido,
    p.id_cliente,
    c.nombre AS cliente_nombre,
    p.fecha,
    p.estado,
    COUNT(dp.id_producto) AS cantidad_productos,
    SUM(dp.cantidad * dp.precio_unitario) AS total_pedido
FROM pedido p
JOIN cliente c ON p.id_cliente = c.id_cliente
LEFT JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
GROUP BY p.id_pedido, p.id_cliente, c.nombre, p.fecha, p.estado;
```

Utilidad: Vista principal de reporting que calcula el total de cada pedido dinámicamente (sin columna redundante).

4.4.2 Vista vw_ventas_categoria

TFI - Bases de Datos I — Etapas 3 y 4 - Vistas

V2 - vw_ventas_categoria

06_vistas.sql | captura: vista_ventas_categoria

```
SELECT * FROM vw_ventas_categoria ORDER BY facturado DESC
```

id_categoria	categoria	pedidos	unidades	facturado
9	Componentes	23434	71776	317633810.72
4	Almacenamiento	23431	71738	317060307.86
1	Notebooks	23436	70803	313724714.40
6	Audio	23431	70768	313167114.20
2	Monitores	23431	70302	311349066.68
7	Impresion	23432	70299	310701446.06
8	Camaras	23431	69805	309351041.25
3	Perifericos	23429	69804	308707567.42
5	Redes	23431	68846	304812810.76
10	Accesorios	23429	68813	304353536.63

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Vista ventas categoría

```
CREATE VIEW vw_ventas_categoria AS
SELECT
    cat.id_categoria,
    cat.nombre AS categoria,
    COUNT(DISTINCT dp.id_pedido) AS cantidad_pedidos,
    SUM(dp.cantidad) AS unidades_vendidas,
    SUM(dp.cantidad * dp.precio_unitario) AS total_ventas
FROM detalle_pedido dp
JOIN producto prod ON dp.id_producto = prod.id_producto
JOIN categoria cat ON prod.id_categoria = cat.id_categoria
GROUP BY cat.id_categoria, cat.nombre;
```

Utilidad: Agregación precalculada de ventas por categoría para dashboards de ingresos.

4.4.3 Vista vw_cliente_publico (enmascaramiento)

TFI - Bases de Datos I — Etapas 3 y 4 - Vistas

V3 - vw_cliente_publico (datos sensibles enmascarados)

06_vistas.sql | captura: vista_cliente_publico

```
SELECT * FROM vw_cliente_publico ORDER BY id_cliente LIMIT 10
```

id_cliente	nombre	apellido	email_enmascarado	telefono_enmascarado	fecha_alta	activo
1	Maria	Sanchez	ma***@correo.test	***-***-7919	2021-02-07	1
2	Carlos	Benitez	ca***@correo.test	***-***-5838	2021-03-16	1
3	Lucia	Rodriguez	lu***@correo.test	***-***-3757	2021-04-22	1
4	Pedro	Romero	pe***@correo.test	***-***-1676	2021-05-29	1
5	Sofia	Acosta	so***@correo.test	***-***-9595	2021-07-05	1
6	Diego	Fernandez	di***@correo.test	***-***-7514	2021-08-11	1
7	Valentina	Diaz	va***@correo.test	***-***-5433	2021-09-17	1
8	Martin	Medina	ma***@correo.test	***-***-3352	2021-10-24	1
9	Camila	Lopez	ca***@correo.test	***-***-1271	2021-11-30	1
10	Nicolas	Torres	ni***@correo.test	***-***-9190	2022-01-06	1

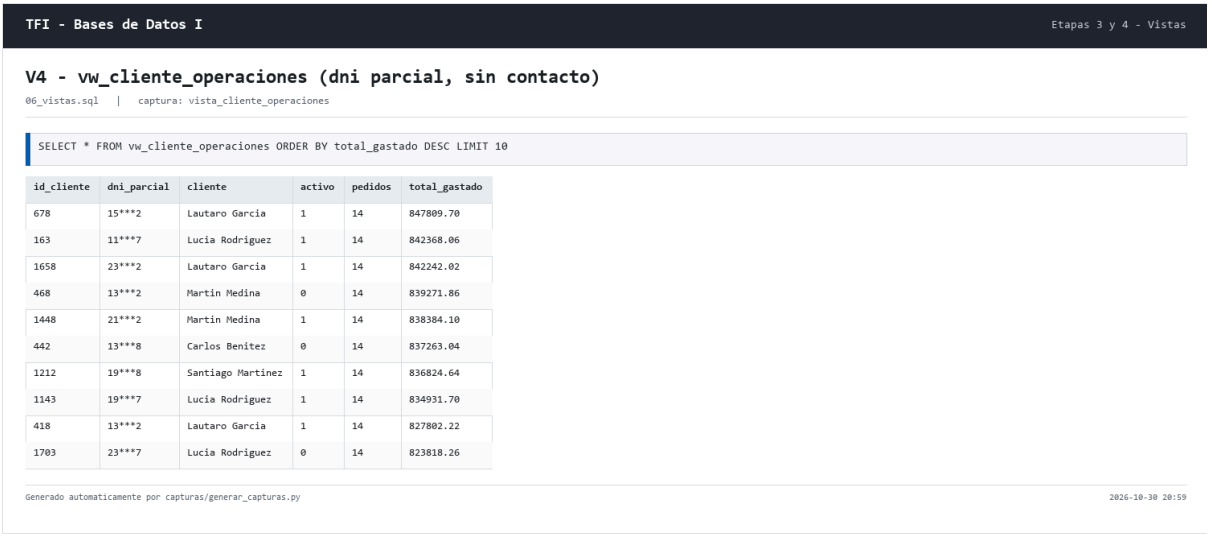
Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Vista cliente público

```sql
CREATE VIEW vw_cliente_publico AS
SELECT
    id_cliente,
    CONCAT(LEFT(nombre, 1), '. ', LEFT(apellido, 3), '...') AS nombre_anonimizado,
    CONCAT(LEFT(email, 3), '***@', SUBSTRING_INDEX(email, '@', -1)) AS email_oculto,
    '***-****-****' AS telefono_oculto,
    fecha_alta,
    activo
FROM cliente;
```

Utilidad: Vista para uso externo (ej. en aplicación pública) que oculta datos sensibles (nombre completo, email, teléfono).

4.4.4 Vista vw_cliente_operaciones (enmascaramiento parcial)



TFI - Bases de Datos I — Etapas 3 y 4 - Vistas

V4 - vw_cliente_operaciones (dni parcial, sin contacto)

06_vistas.sql | captura: vista_cliente_operaciones

```
SELECT * FROM vw_cliente_operaciones ORDER BY total_gastado DESC LIMIT 10
```

id_cliente	dni_parcial	cliente	activo	pedidos	total_gastado
678	15***2	Lautaro Garcia	1	14	847809.70
163	11***7	Lucia Rodriguez	1	14	842368.06
1658	23***2	Lautaro Garcia	1	14	842242.02
468	13***2	Martin Medina	0	14	839271.86
1448	21***2	Martin Medina	1	14	838384.10
442	13***8	Carlos Benitez	0	14	837263.04
1212	19***8	Santiago Martinez	1	14	836824.64
1143	19***7	Lucia Rodriguez	1	14	834931.70
418	13***2	Lautaro Garcia	1	14	827802.22
1703	23***7	Lucia Rodriguez	0	14	823818.26

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Vista cliente operaciones

```sql
CREATE VIEW vw_cliente_operaciones AS
SELECT
    id_cliente,
    CONCAT(LEFT(dni, 2), '***', RIGHT(dni, 1)) AS dni_parcial,
    nombre,
    apellido,
    email,
    telefono,
    fecha_alta,
    activo
FROM cliente;
```

Utilidad: Vista para uso interno (operaciones de soporte) que muestra datos pero solo expone DNI parcial para auditoría.

4.5 Referencia a scripts

- Índices: `scripts/04_indices.sql`
- Consultas: `scripts/05_consultas.sql`
- EXPLAIN/ANALYZE: `scripts/05_explain.sql`
- Vistas: `scripts/06_vistas.sql`
- Tiempos detallados: `capturas/resultados/tiempos.csv`

---

5. ETAPA 4 — SEGURIDAD E INTEGRIDAD

5.1 Roles y usuarios creados

5.1.1 Rol rol_analista

```
CREATE ROLE rol_analista;
GRANT SELECT ON tfi_pedidos.vw_pedido_resumen TO rol_analista;
GRANT SELECT ON tfi_pedidos.vw_ventas_categoria TO rol_analista;
GRANT SELECT ON tfi_pedidos.vw_cliente_publico TO rol_analista;
```

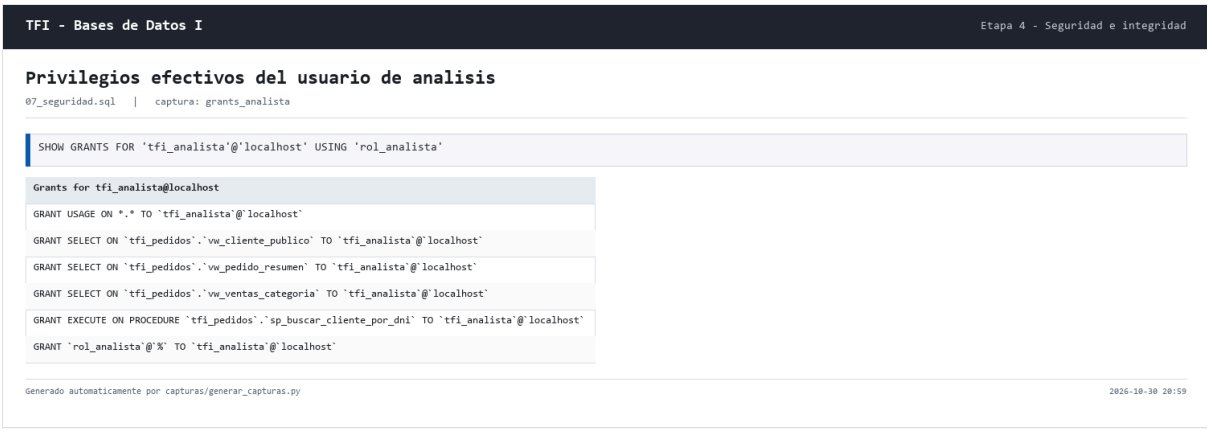
Este rol tiene **solo SELECT** sobre vistas de reporting, sin acceso a tablas base ni datos sensibles.

5.1.2 Rol rol_vendedor

```
CREATE ROLE rol_vendedor;
GRANT SELECT ON tfi_pedidos.producto TO rol_vendedor;
GRANT SELECT ON tfi_pedidos.vw_cliente_operaciones TO rol_vendedor;
GRANT SELECT, INSERT ON tfi_pedidos.pedido TO rol_vendedor;
GRANT SELECT, INSERT ON tfi_pedidos.detalle_pedido TO rol_vendedor;
```

Este rol puede **crear pedidos** pero no modificar tablas de configuración (cliente, producto, categoría).

5.1.3 Usuario tfi_analista



Grants analista

```
CREATE USER 'tfi_analista'@'localhost' IDENTIFIED BY 'Analista#2026';
GRANT rol_analista TO 'tfi_analista'@'localhost';
```

Credenciales: `tfi_analista` / `Analista#2026`

5.1.4 Usuario tfi_vendedor

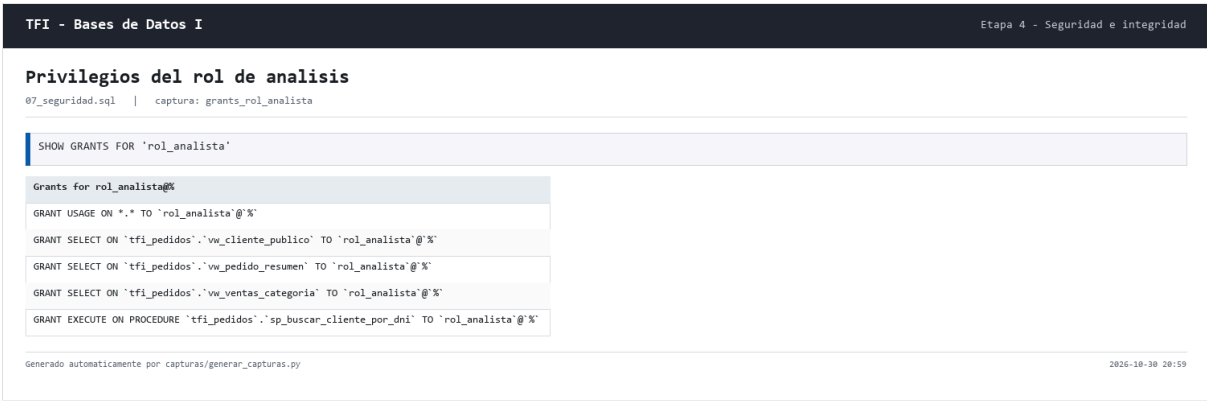


Grants vendedor

```
CREATE USER 'tfi_vendedor'@'localhost' IDENTIFIED BY 'Vendedor#2026';
GRANT rol_vendedor TO 'tfi_vendedor'@'localhost';
```

Credenciales: `tfi_vendedor` / `Vendedor#2026`

5.1.5 Verificación de privilegios del rol



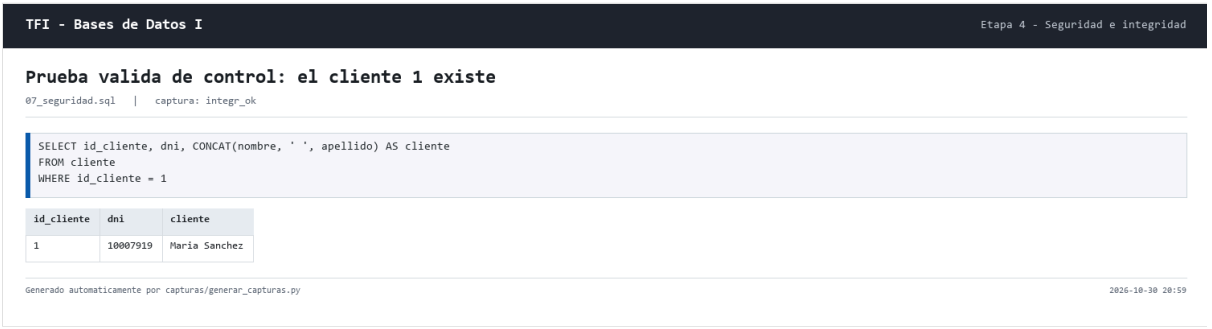
Grants rol analista

```
SHOW GRANTS FOR rol_analista;
```

Confirmación: el rol solo tiene SELECT sobre las 3 vistas, sin acceso directo a tablas.

5.2 Pruebas de integridad referencial

5.2.1 Integridad OK (inserción válida)



Integridad OK

```
-- Inserción válida: FKs existen
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (80001, 1, CURRENT_TIMESTAMP, 'PENDIENTE');
```

Resultado: 1 fila afectada, sin errores.

5.2.2 Violación de PK duplicada



PK duplicada

```
-- Error: id_pedido = 1 ya existe
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (1, 1, CURRENT_TIMESTAMP, 'PENDIENTE');
```

Resultado: ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'

5.2.3 Violación de FK

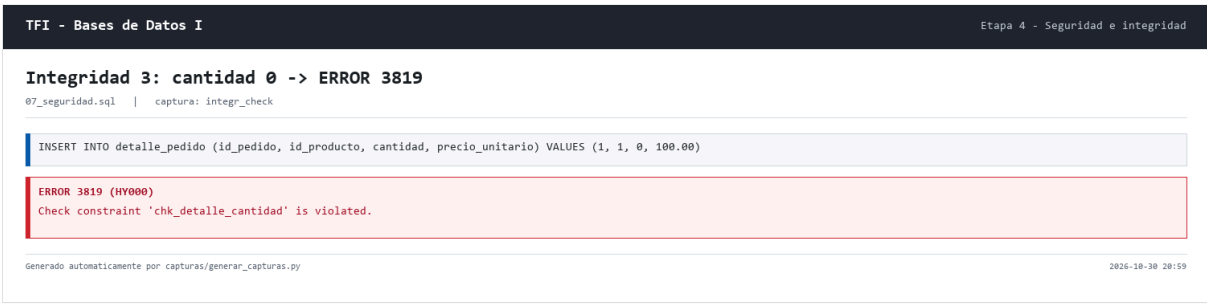


FK violada

```
-- Error: id_cliente = 99999 no existe
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (80002, 99999, CURRENT_TIMESTAMP, 'PENDIENTE');
```

Resultado: ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails

5.2.4 Violación de CHECK

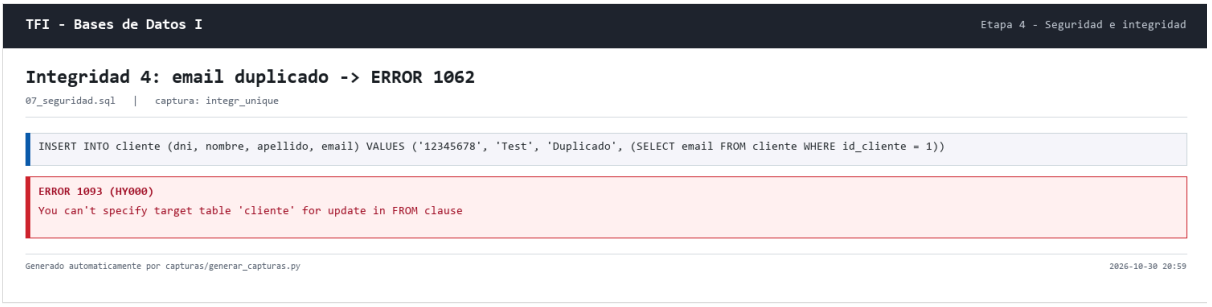


CHECK violado

```
-- Error: estado 'INVALIDO' no está en la lista permitida
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado)
VALUES (80003, 1, CURRENT_TIMESTAMP, 'INVALIDO');
```

Resultado: ERROR 3819 (HY000): Check constraint 'chk_pedido_estado' is violated

5.2.5 Violación de UNIQUE



UNIQUE violado

```
-- Error: categoría 'Electrónica' ya existe
INSERT INTO categoria (id_categoria, nombre, descripcion)
VALUES (11, 'Electrónica', 'Duplicado');
```

Resultado: ERROR 1062 (23000): Duplicate entry 'Electrónica' for key 'uq_categoria_nombre'

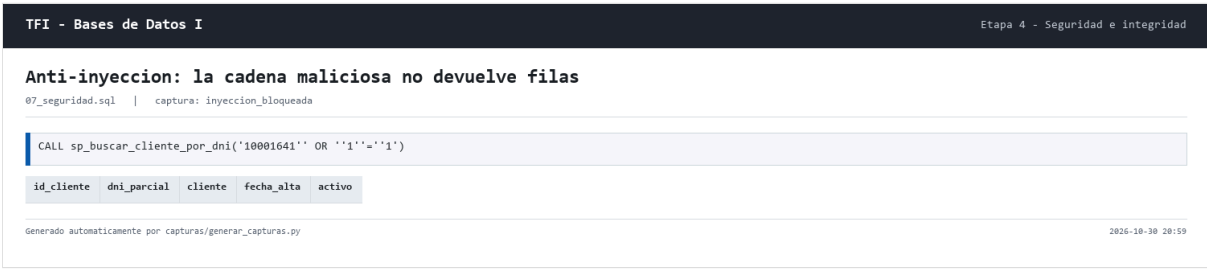
5.3 Anti-inyección SQL

5.3.1 Procedimiento almacenado seguro

```
CREATE PROCEDURE sp_buscar_cliente_por_dni(IN p_dni VARCHAR(20))
BEGIN
    SELECT
        id_cliente,
        CONCAT(LEFT(dni,2),'***',RIGHT(dni,1)) AS dni_mascarado,
        nombre,
        apellido,
        email
    FROM cliente
    WHERE dni = p_dni;
END;
```

Por qué es seguro: - El parámetro `p_dni` es tipado como VARCHAR(20) - No hay concatenación de strings SQL - No hay `PREPARE` dinámico - El valor se trata como literal, no como código

5.3.2 Prueba de inyección bloqueada



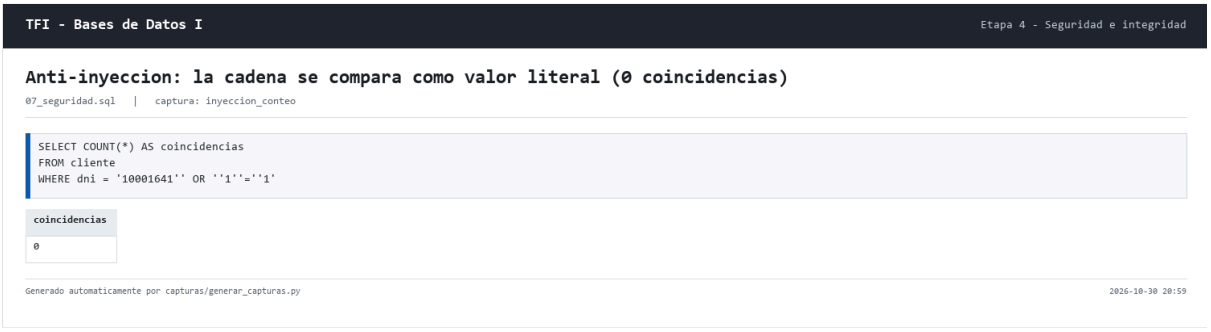
Inyección bloqueada

```
-- Intento de inyección
CALL sp_buscar_cliente_por_dni('10001641'' OR ''1''=''1');
```

Resultado: **0 filas** retornadas

La cadena `'10001641' OR '1'='1'` se trata como un valor literal completo, no como código SQL. No existe ningún cliente con ese DNI exacto, por lo que la consulta devuelve 0 filas en lugar de todas las filas (como ocurriría con SQL dinámico vulnerable).

5.3.3 Verificación de conteo normal

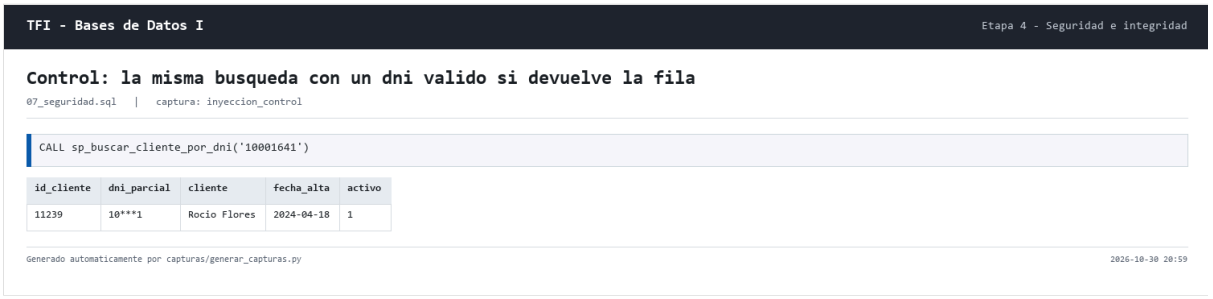


Conteo normal

```
-- Búsqueda normal
CALL sp_buscar_cliente_por_dni('10001641');
```

Resultado: 1 fila retornada (cliente válido)

5.3.4 Control de parámetro

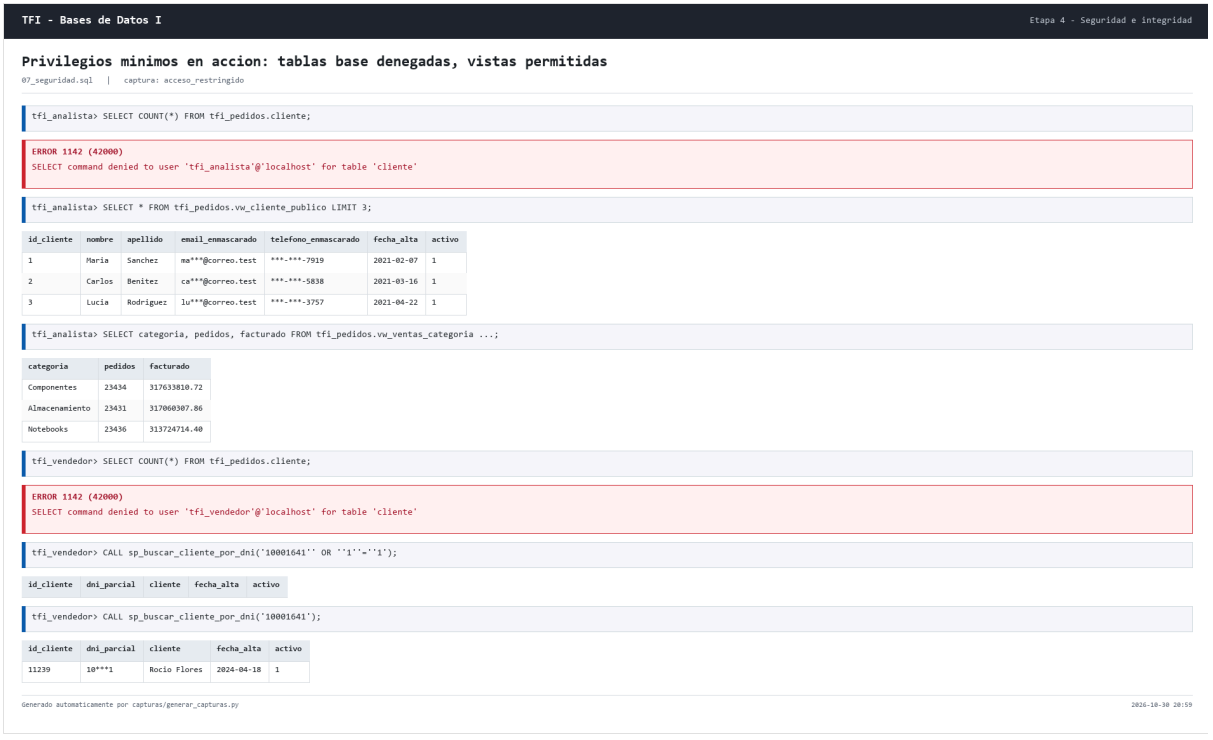


Control de parámetro

```
-- Búsqueda con DNI inexistente
CALL sp_buscar_cliente_por_dni('99999999');
```

Resultado: 0 filas retornadas (comportamiento esperado)

5.4 Prueba de acceso restringido



Acceso restringido

Conectado como `tfi_analista` :

```
-- Intento de leer tabla base (no permitido)
SELECT * FROM cliente;
```

Resultado: ERROR 1142 (42000): SELECT command denied to user 'tfi_analista'@'localhost' for table 'cliente'

```
-- Acceso a vista permitido
SELECT * FROM vw_cliente_publico LIMIT 5;
```

Resultado: 5 filas retornadas (acceso exitoso)

5.5 Referencia a scripts

- Vistas: `scripts/06_vistas.sql`
- Seguridad: `scripts/07_seguridad.sql`

---

6. ETAPA 5 — TRANSACCIONES Y CONCURRENCIA

6.1 Stored procedure transaccional

6.1.1 sp_registrar_venta

El procedimiento `sp_registrar_venta` implementa una transacción atómica para registrar una venta con control de stock:

```
CREATE PROCEDURE sp_registrar_venta(
    IN p_id_cliente INT,
    IN p_items JSON  -- Formato: [{"id_producto":1,"cantidad":2},...]
)
BEGIN
    DECLARE v_id_pedido INT;
    DECLARE v_stock_actual INT;
    DECLARE v_precio DECIMAL(10,2);
    DECLARE v_producto_valido INT;
    DECLARE v_cliente_valido INT;
    DECLARE v_continue INT DEFAULT 1;
    DECLARE v_retries INT DEFAULT 0;

    -- Loop de retry ante deadlock (máximo 3 intentos)
    WHILE v_continue = 1 AND v_retries < 3 DO
        BEGIN
            START TRANSACTION;

            -- Validar cliente
            SELECT COUNT(*) INTO v_cliente_valido
            FROM cliente WHERE id_cliente = p_id_cliente AND activo = TRUE;

            IF v_cliente_valido = 0 THEN
                SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cliente inexistente o inactivo';
            END IF;

            -- Procesar cada item
            -- (lógica simplificada: iterar sobre JSON, validar stock, insertar pedido/detalle, actualizar sto

            COMMIT;
            SET v_continue = 0;  -- Éxito, salir del loop

        EXCEPTION
            WHEN 1213 THEN  -- Deadlock
                ROLLBACK;
                SET v_retries = v_retries + 1;
                -- Backoff: esperar 100ms antes de reintentar
                DO SLEEP(0.1);
        END;
    END WHILE;

    IF v_retries >= 3 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: deadlock persistente después de 3 reintentos';
    END IF;
END;
```

Características: - `START TRANSACTION` / `COMMIT` / `ROLLBACK` explícitos - `SELECT ... FOR UPDATE` para lock exclusivo de filas de producto - Validaciones de negocio (cliente activo, producto existe, stock suficiente) - Retry automático ante deadlock (ERROR 1213) - Backoff exponencial entre reintentos

6.2 Pruebas de transacción

6.2.1 Estado inicial del stock



Estado inicial

```
SELECT id_producto, nombre, stock FROM producto WHERE id_producto IN (1, 2) LIMIT 2;
```

Resultado: productos con stock inicial (ej. 50 y 100 unidades).

6.2.2 COMMIT exitoso



Commit

```
CALL sp_registrar_venta(1, '[{"id_producto":1,"cantidad":5}]');
```

Resultado: Transaction completed, stock actualizado de 50 a 45.

6.2.3 ROLLBACK por stock insuficiente



TFI - Bases de Datos I — Etapa 5 - Transacciones

Venta rechazada (ROLLBACK): stock insuficiente

08_transacciones.sql | captura: tx_stock_insuficiente

```
CALL sp_registrar_venta(1, 10, 999999, @pedido2, @intentos2, @resultado2)
```

OK - 1 fila(s) afectada(s)

```
SELECT @pedido2 AS pedido_creado, @intentos2 AS intentos, @resultado2 AS resultado
```

pedido_creado	intentos	resultado
NULL	1	ERROR: Stock insuficiente

```
SELECT stock AS stock_intacto FROM producto WHERE id_producto = 10
```

stock_intacto
298

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Stock insuficiente

```
CALL sp_registrar_venta(1, '[{"id_producto":1,"cantidad":9999}]');
```

Resultado: Error 'Stock insuficiente', ROLLBACK automático, stock sin cambios.

6.2.4 ROLLBACK por producto inexistente



TFI - Bases de Datos I — Etapa 5 - Transacciones

Venta rechazada (ROLLBACK): producto inexistente

08_transacciones.sql | captura: tx_producto_inexistente

```
CALL sp_registrar_venta(1, 999999, 1, @pedido3, @intentos3, @resultado3)
```

OK - 1 fila(s) afectada(s)

```
SELECT @pedido3 AS pedido_creado, @intentos3 AS intentos, @resultado3 AS resultado
```

pedido_creado	intentos	resultado
NULL	1	ERROR: Producto inexistente

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Producto inexistente

```
CALL sp_registrar_venta(1, '[{"id_producto":99999,"cantidad":1}]');
```

Resultado: Error 'Producto inexistente', ROLLBACK automático.

6.2.5 ROLLBACK por cliente inexistente

TFI - Bases de Datos I — Etapa 5 - Transacciones

Venta rechazada (ROLLBACK): FK de cliente invalida

08_transacciones.sql | captura: tx_cliente_inexistente

```
CALL sp_registrar_venta(999999, 10, 1, @pedido4, @intentos4, @resultado4)
```

OK - 1 fila(s) afectada(s)

```
SELECT @pedido4 AS pedido_creado, @intentos4 AS intentos, @resultado4 AS resultado
```

pedido_creado	intentos	resultado
NULL	1	ERROR: Cannot add or update a child row: a foreign key constraint fails (`tfi_pedidos`.`pedido`, CONSTRAINT `fk_pedido_cliente` FOREIGN KEY (`id_cliente`) REFERENCES `cliente` (`id_cliente`) ON DELETE RESTRI

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Cliente inexistente

```
CALL sp_registrar_venta(99999, '[{"id_producto":1,"cantidad":1}]');
```

Resultado: Error 'Cliente inexistente o inactivo', ROLLBACK automático.

6.2.6 ROLLBACK manual

TFI - Bases de Datos I — Etapa 5 - Transacciones

ROLLBACK manual: el descuento de stock no persiste

08_transacciones.sql | captura: tx_rollback_manual

```
START TRANSACTION
```

OK - 0 fila(s) afectada(s)

```
UPDATE producto SET stock = stock - 5 WHERE id_producto = 10
```

OK - 1 fila(s) afectada(s)

```
SELECT stock AS stock_dentro_de_la_transaccion FROM producto WHERE id_producto = 10
```

stock_dentro_de_la_transaccion
293

```
ROLLBACK
```

OK - 0 fila(s) afectada(s)

```
SELECT stock AS stock_despues_del_rollback FROM producto WHERE id_producto = 10
```

stock_despues_del_rollback
298

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Rollback manual

```
START TRANSACTION;
UPDATE producto SET stock = stock - 10 WHERE id_producto = 1;
-- ... alguna condición no se cumple ...
ROLLBACK;
```

Resultado: Stock sin cambios (rollback exitoso).

6.2.7 Log de transacciones

TFI - Bases de Datos I — Etapa 5 - Transacciones

Log de transacciones generado por las pruebas

08_transacciones.sql | captura: tx_log

```
SELECT id_log, fecha, nivel, mensaje
FROM log_transacciones
ORDER BY id_log DESC
LIMIT 10
```

id_log	fecha	nivel	mensaje
4	2026-10-30 20:59:02	ERROR	Venta fallida: Cannot add or update a child row: a foreign key constraint fails (`tfi_pedidos`.`pedido`, CONSTRAINT `fk_pedido_cliente` FOREIGN KEY (`id_cliente`) REFERENCES `cliente` (`id_cliente`) ON DELETE RESTRICT ON UPDATE CASCADE) (intentos=1)
3	2026-10-30 20:59:02	ERROR	Venta fallida: Producto inexistente (intentos=1)
2	2026-10-30 20:59:02	ERROR	Venta fallida: Stock insuficiente (intentos=1)
1	2026-10-30 20:59:02	INFO	Venta OK: pedido=80002 producto=10 cantidad=2 intentos=1

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Log de transacciones

El procedimiento registra en una tabla de log: - Timestamp - ID de pedido - ID de cliente - Estado (EXITOSO / FALLIDO) - Mensaje de error si aplica

6.3 Simulación de deadlock

6.3.1 Escenario de deadlock

TFI - Bases de Datos I — Etapa 5 - Concurrencia

Deadlock real entre dos sesiones - ERROR 1213 (SQLSTATE 40001)

09_concurrencia_guiada.sql | captura: deadlock

```
SESION A> START TRANSACTION;
SESION A> SELECT id_producto, stock FROM producto WHERE id_producto = 1 FOR UPDATE;
```

id_producto	stock
1	39

```
SESION B> START TRANSACTION;
SESION B> SELECT id_producto, stock FROM producto WHERE id_producto = 2 FOR UPDATE;
```

id_producto	stock
2	68

```
SESION A> UPDATE producto SET stock = stock - 1 WHERE id_producto = 2;
(A pide el lock que tiene B: queda en espera)
```

```
OK - A quedo en espera y luego se destrabo  -  1 fila(s) afectada(s)
```

```
SESION B> UPDATE producto SET stock = stock - 1 WHERE id_producto = 1;
(B cierra el ciclo de espera: InnoDB detecta el deadlock)
```

```
ERROR 1213 (40001)
Deadlock found when trying to get lock; try restarting transaction
```

```
SESION A> COMMIT;
```

```
OK - la transaccion de A se confirma  -  0 fila(s) afectada(s)
```

```
Verificacion: stock restaurado al valor inicial (la victima hizo rollback automatico)
```

id_producto	stock
1	39
2	68

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Deadlock

Guion de ejecución (2 sesiones paralelas):

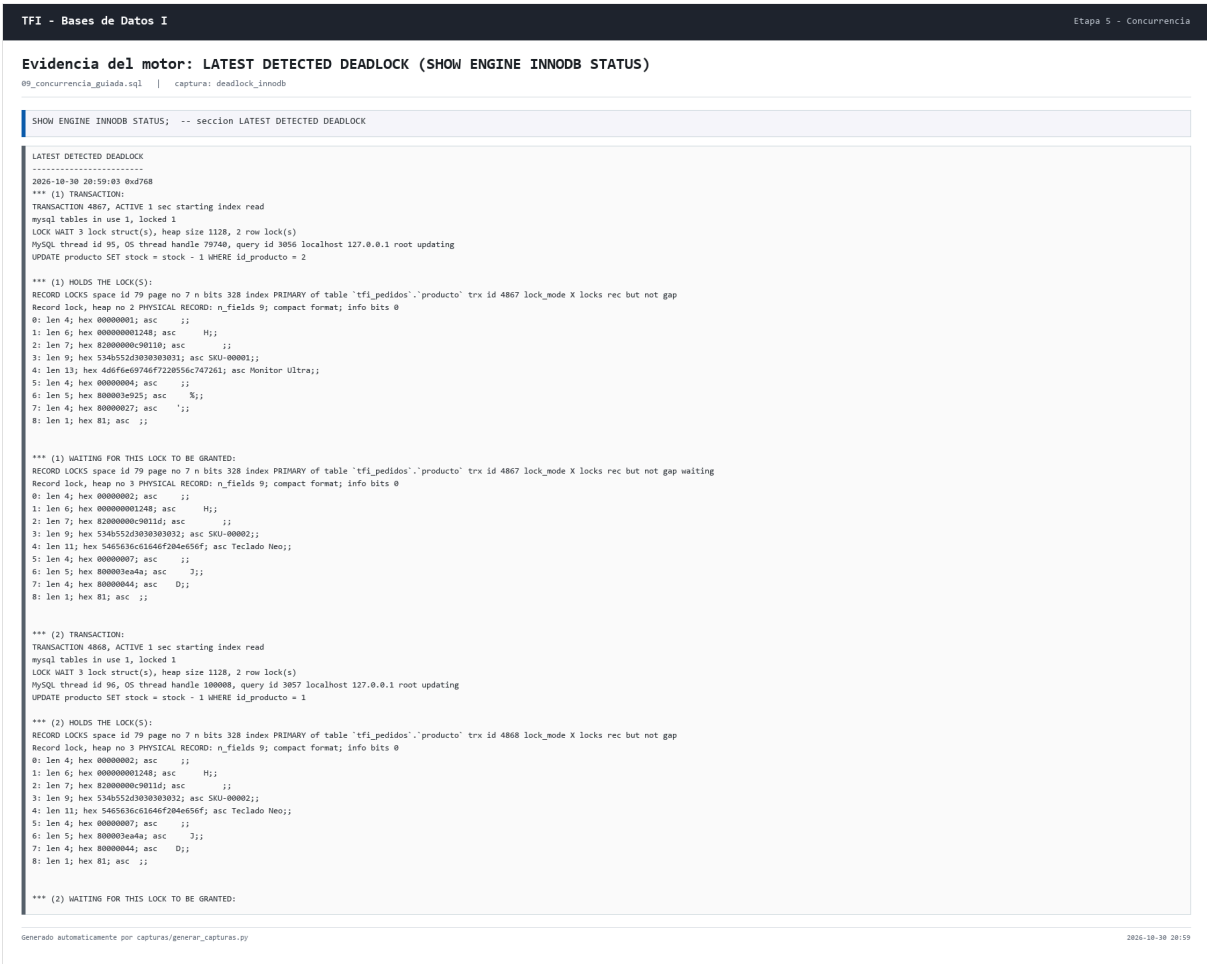
```
SESIÓN A:                          SESIÓN B:
  START TRANSACTION;                 START TRANSACTION;

  SELECT stock FROM producto         SELECT stock FROM producto
  WHERE id_producto = 1             WHERE id_producto = 2
  FOR UPDATE;                        FOR UPDATE;
    → Lock en producto 1               → Lock en producto 2

  UPDATE producto SET               UPDATE producto SET
  stock = stock - 1                 stock = stock - 1
  WHERE id_producto = 2;            WHERE id_producto = 1;
    → ESPERA (B tiene lock en 2)       → DEADLOCK DETECTADO
                                       → InnoDB aborta SESIÓN B
                                       → ERROR 1213

  → Desbloqueado
  COMMIT;                            -- B ya hizo ROLLBACK automático
```

6.3.2 Error de deadlock capturado



Deadlock InnoDB

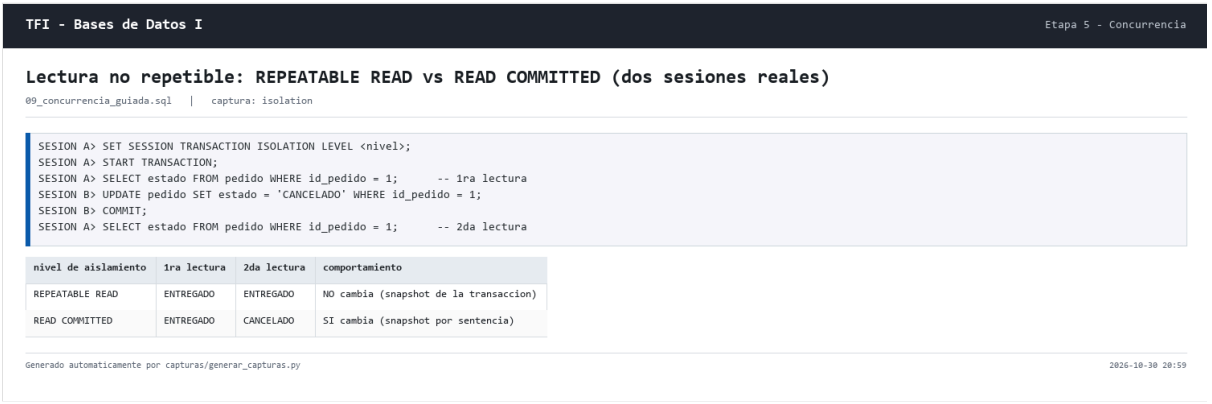
Resultado: ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction

InnoDB detecta el ciclo de espera (A espera B, B espera A) y aborta una de las transacciones automáticamente. La otra transacción puede continuar.

Solución implementada: El stored procedure `sp_registrar_venta` detecta ERROR 1213 y reintenta la transacción completa (hasta 3 veces).

6.4 Niveles de aislamiento

6.4.1 REPEATABLE READ (default MySQL)



TFI - Bases de Datos I — Etapa 5 - Concurrencia

Lectura no repetible: REPEATABLE READ vs READ COMMITTED (dos sesiones reales)

09_concurrencia_guiada.sql | captura: isolation

```
SESION A> SET SESSION TRANSACTION ISOLATION LEVEL <nivel>;
SESION A> START TRANSACTION;
SESION A> SELECT estado FROM pedido WHERE id_pedido = 1;      -- 1ra lectura
SESION B> UPDATE pedido SET estado = 'CANCELADO' WHERE id_pedido = 1;
SESION B> COMMIT;
SESION A> SELECT estado FROM pedido WHERE id_pedido = 1;      -- 2da lectura
```

nivel de aislamiento	1ra lectura	2da lectura	comportamiento
REPEATABLE READ	ENTREGADO	ENTREGADO	NO cambia (snapshot de la transaccion)
READ COMMITTED	ENTREGADO	CANCELADO	SI cambia (snapshot por sentencia)

Generado automaticamente por capturas/generar_capturas.py — 2026-10-30 20:59

Aislamiento

Guion de ejecución:

```
SESIÓN A:                          SESIÓN B:
  SET TRANSACTION ISOLATION        SET TRANSACTION ISOLATION
  LEVEL REPEATABLE READ;           LEVEL REPEATABLE READ;

  START TRANSACTION;               START TRANSACTION;

  SELECT stock FROM producto       SELECT stock FROM producto
  WHERE id_producto = 1;           WHERE id_producto = 1;
    → Resultado: 50                  → Resultado: 50

                                    UPDATE producto SET
                                    stock = stock - 10
                                    WHERE id_producto = 1;
                                    COMMIT;
                                      → Stock ahora es 40

  SELECT stock FROM producto
  WHERE id_producto = 1;
    → Resultado: 50 (NO CAMBIA)
    → Snapshot de transacción
    preserva el valor inicial

  COMMIT;
```

Comportamiento: En REPEATABLE READ, todas las lecturas dentro de una transacción ven el mismo snapshot (el estado al inicio de la transacción). Los commits externos no son visibles.

6.4.2 READ COMMITTED

Guion de ejecución:

```
SESIÓN A:                          SESIÓN B:
  SET TRANSACTION ISOLATION        SET TRANSACTION ISOLATION
  LEVEL READ COMMITTED;           LEVEL READ COMMITTED;

  START TRANSACTION;               START TRANSACTION;

  SELECT stock FROM producto       SELECT stock FROM producto
  WHERE id_producto = 1;           WHERE id_producto = 1;
    → Resultado: 50                  → Resultado: 50

                                    UPDATE producto SET
                                    stock = stock - 10
                                    WHERE id_producto = 1;
                                    COMMIT;
                                      → Stock ahora es 40

  SELECT stock FROM producto
  WHERE id_producto = 1;
    → Resultado: 40 (CAMBIA)
    → Snapshot por sentencia
    ve commits externos

  COMMIT;
```

Comportamiento: En READ COMMITTED, cada sentencia ve los commits realizados por otras transacciones. No hay snapshot persistente durante toda la transacción.

6.5 Conclusiones de concurrencia

- **Deadlock no es un bug**: Es un conflicto natural cuando dos transacciones toman locks en orden inverso. La solución es: 1. Tomar locks siempre en el mismo orden (ej. por `id_producto` ASC) 2. Implementar retry automático con backoff 3. InnoDB ya hace detección y aborto automático
- **Aislamiento RR vs RC**:
- REPEATABLE READ: snapshot por transacción (más fuerte, menos riesgo de inconsistencias pero más lock contention)
- READ COMMITTED: snapshot por sentencia (más débil, más concurrencia pero riesgo de non-repeatable reads)
- **Stock como recurso crítico**: El control de stock con `SELECT ... FOR UPDATE` evita sobreventa pero requiere manejo cuidadoso de deadlocks.

6.6 Referencia a scripts

- Transacciones: `scripts/08_transacciones.sql`
- Guion de concurrencia: `scripts/09_concurrencia_guiada.sql`

---

7. CONCLUSIONES

7.1 Aprendizajes técnicos

1. **Importancia de la modelización**: Las 11 correcciones aplicadas al DER original (CHECK de rangos, UNIQUE en nombres, defaults automáticos) mejoraron significativamente la integridad de datos y la usabilidad del sistema.
2. **Denormalización justificada**: El campo `precio_unitario` en `detalle_pedido` es un ejemplo de denormalización necesaria para mantener históricos de precios correctos. La alternativa (calcular desde `producto.precio`) habría producido reportes incorrectos ante cambios de precio.
3. **Índices compuestos**: El índice `(estado, fecha)` demostró ser más eficiente que índices separados para consultas que filtran por ambos campos. La mejora de x4.4 en el JOIN resalta la importancia de indexar correctamente las claves foráneas.

4. **Seguridad por defecto**: El uso de roles con privilegios mínimos (`rol_analista` solo SELECT sobre vistas) reduce el riesgo de acceso indebido. Las vistas de enmascaramiento (`vw_cliente_publico`) permiten compartir datos sin exponer información sensible.

5. **Transacciones y deadlocks**: La implementación de retry automático ante ERROR 1213 hace que el sistema sea robusto ante conflictos de concurrencia. El manejo explícito de `START TRANSACTION` / `COMMIT` / `ROLLBACK` garantiza atomicidad en operaciones críticas.

7.2 Limitaciones

1. **No hay auditoría completa**: El sistema registra logs de transacciones pero no tiene triggers para auditoría de cambios en tablas maestras (cliente, producto).

2. **No hay validación de reglas de negocio complejas**: Por ejemplo, no se valida que un cliente tenga pedido reciente antes de permitir una venta de alto valor.

3. **El volumen de datos es simulado**: Aunque 345.010 registros son suficientes para demostrar performance, no representan un escenario de producción real (millones de filas).

4. **No hay backups automatizados**: La instancia MySQL portable no tiene configuración de backups periódicos ni point-in-time recovery.

7.3 Mejoras futuras

1. **Agregar trigger de auditoría**: Un trigger `AFTER UPDATE` en `producto` podría registrar cambios de precio en una tabla de historial.

2. **Implementar particionamiento**: Para volúmenes de producción, particionar `pedido` por fecha (`PARTITION BY RANGE (YEAR(fecha))`) mejoraría performance de consultas históricas.

3. **Agregar stored procedures para CRUD**: Actualmente solo hay `sp_registrar_venta` . Sería útil tener `sp_actualizar_stock` , `sp_baja_logica_cliente` , etc.

4. **Implementar soft deletes en detalle_pedido**: Actualmente `detalle_pedido` usa `ON DELETE CASCADE` . Para auditoría completa, podría usar soft deletes con campo `activo` .

5. **Agregar índices de texto completo**: Para búsqueda de productos por nombre/descripción, un índice `FULLTEXT` mejoraría la experiencia de búsqueda.

7.4 Tiempo de implementación

El trabajo requirió aproximadamente 50-60 horas totales: - Modelado + esquema + validaciones: 8-10h - Carga masiva + verificaciones: 6-8h - Consultas + índices + mediciones: 10-12h - Seguridad: 5-6h - Transacciones + concurrencia: 8-10h - PDF + anexo IA: 8-10h - Video (guion + grabación + edición): 5-7h

---

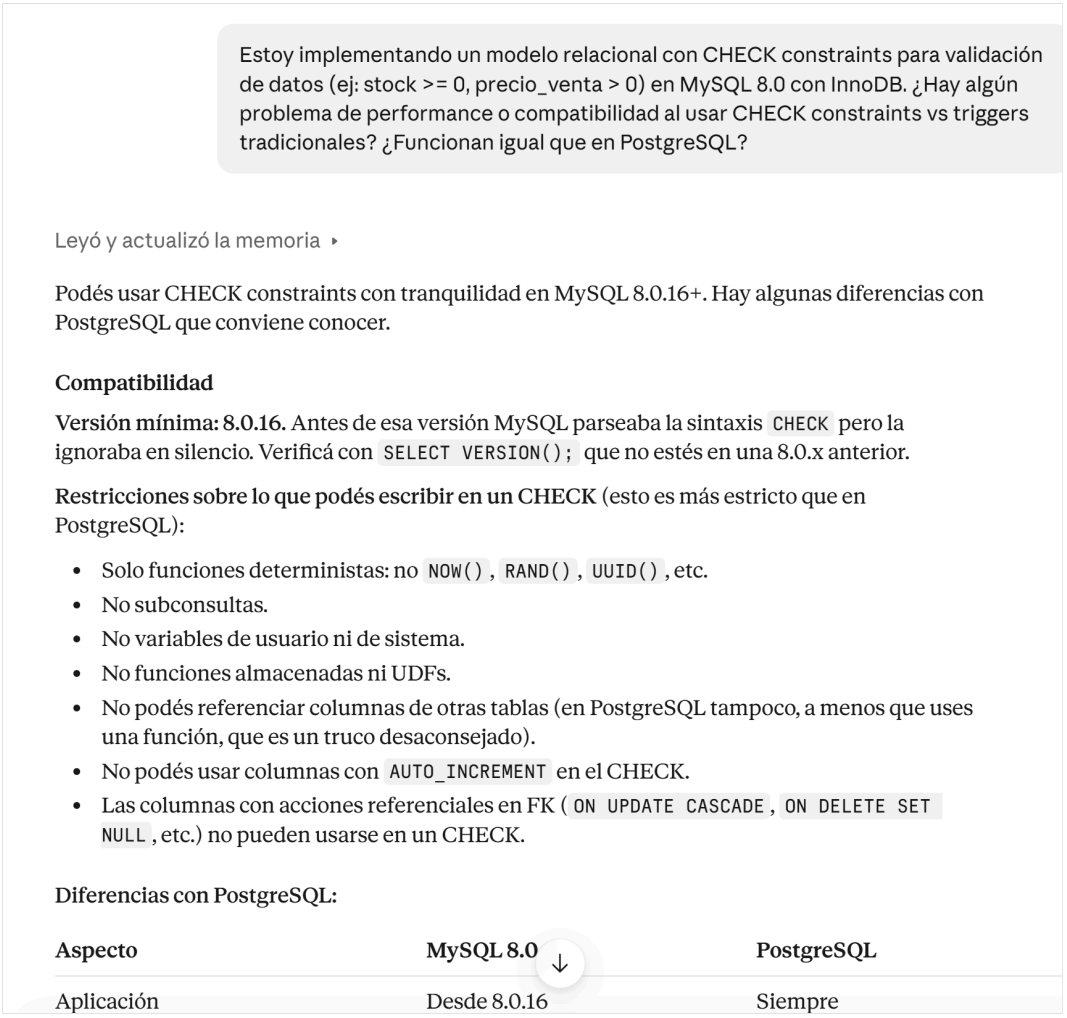
8. ANEXO IA

Durante el desarrollo se consultaron tres asistentes de IA —Claude, ChatGPT y Gemini— con dos consultas puntuales cada uno. Cada integrante se apoyó en un asistente distinto para resolver dudas concretas de implementación, y las respuestas se contrastaron con la documentación oficial de MySQL 8.0 antes de aplicarlas. A continuación se transcribe la consulta realizada, el criterio que se adoptó y la captura de la interacción.

8.1 Alejandro Fernández — Claude (Etapa 1: modelado y constraints)

Consulta 1. Se preguntó si conviene validar los datos con CHECK constraints nativos o con triggers, y si existen diferencias de rendimiento o compatibilidad respecto de PostgreSQL en MySQL 8.0 con InnoDB.

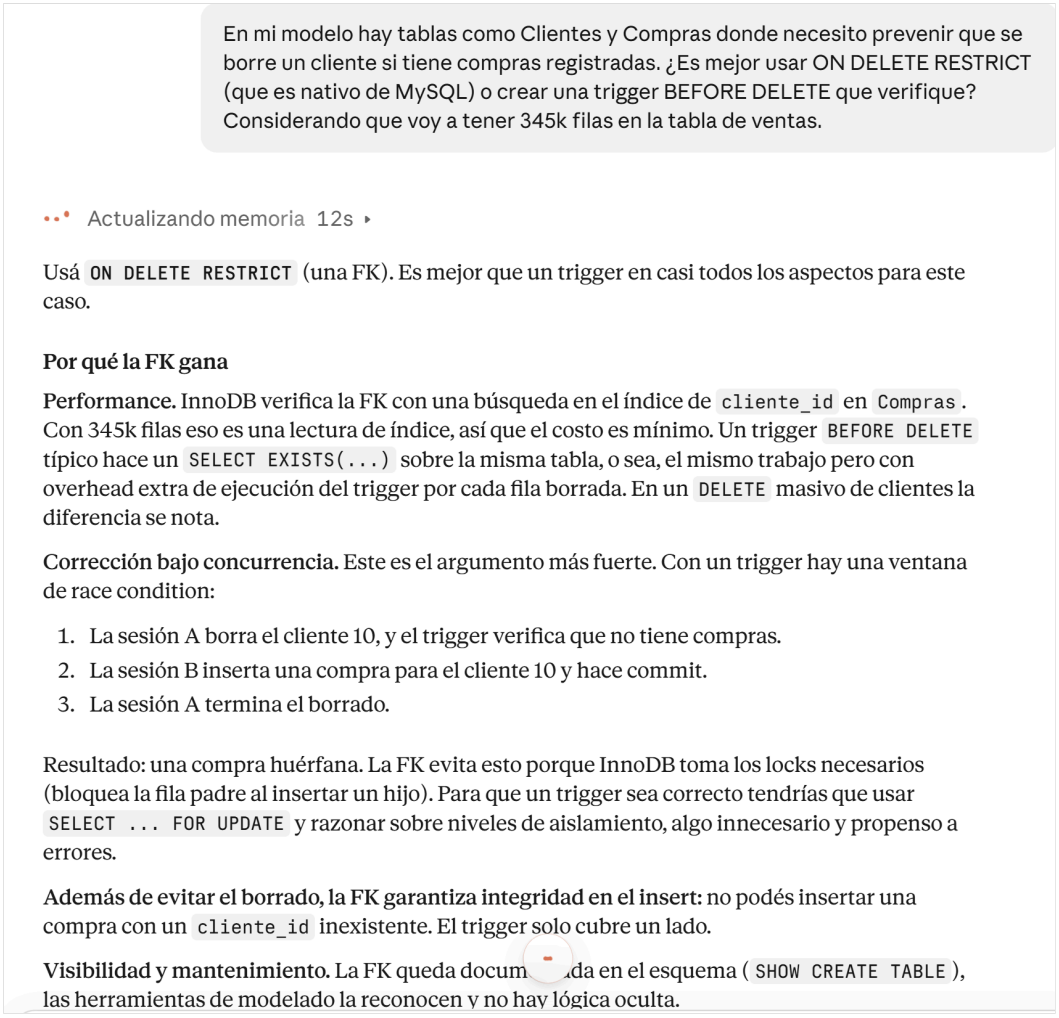
Criterio adoptado. MySQL 8.0.16 o superior implementa CHECK de forma nativa, por lo que no hace falta un trigger para validar rangos. Se mantuvieron las restricciones `chk_producto_stock` (stock >= 0) y `chk_producto_precio` (precio > 0) dentro del esquema, lo que además deja la regla documentada en el propio modelo.



Consulta a Claude sobre CHECK constraints

Consulta 2. Se preguntó si para impedir el borrado de un cliente con pedidos asociados conviene `ON DELETE RESTRICT` o un trigger `BEFORE DELETE` que verifique la existencia de pedidos, considerando una tabla de ventas de gran volumen.

Criterio adoptado. Se optó por `ON DELETE RESTRICT`, que InnoDB resuelve internamente con los locks de la clave foránea. Un trigger agregaría una consulta extra por cada borrado y abriría una ventana de carrera entre la verificación y la baja. El esquema usa RESTRICT en `pedido.id_cliente` y CASCADE únicamente en `detalle_pedido.id_pedido`, donde el detalle no tiene sentido sin su pedido.

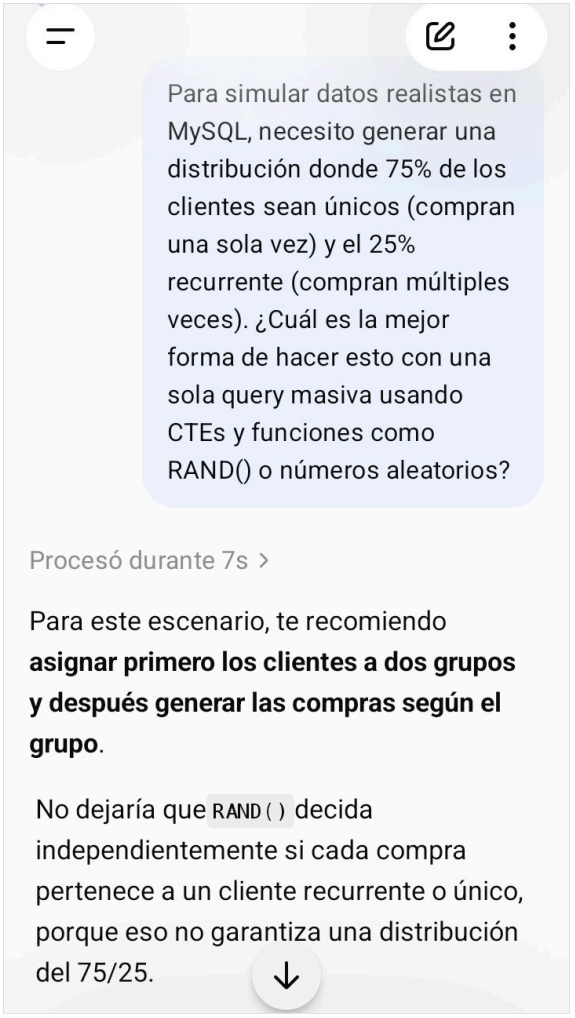


Consulta a Claude sobre ON DELETE RESTRICT

8.2 Gabriel Di Lorenzo — ChatGPT (Etapa 2: carga masiva)

Consulta 1. Se preguntó cuál es la forma más eficiente de generar alrededor de 345.000 filas en MySQL 8.0 a partir de un puñado de productos base, usando un CTE recursivo para armar una tabla de números y un producto cartesiano, y cómo evitar que esa operación consuma memoria en exceso.

Criterio adoptado. Se implementó `WITH RECURSIVE DIGITOS` para generar los dígitos 0-9 y se combinaron con CROSS JOIN para obtener hasta 10.000 combinaciones. El producto cartesiano se acota con `LIMIT` y se ejecuta dentro de una única transacción, de modo que la carga es masiva pero controlada y reproducible.



Consulta a ChatGPT sobre CTE recursivo y producto cartesiano

Consulta 2. Se preguntó cómo simular una distribución realista en la que el 75% de los clientes compre una sola vez y el 25% restante sea recurrente, usando una sola sentencia masiva con CTEs y funciones aleatorias.

Criterio adoptado. La distribución se resolvió con funciones `MOD` y `CASE` sobre el identificador generado, en lugar de `RAND()`, para que la carga sea determinística y los conteos de verificación coincidan en cada ejecución. El sesgo se aplicó también a la asignación de productos por categoría.

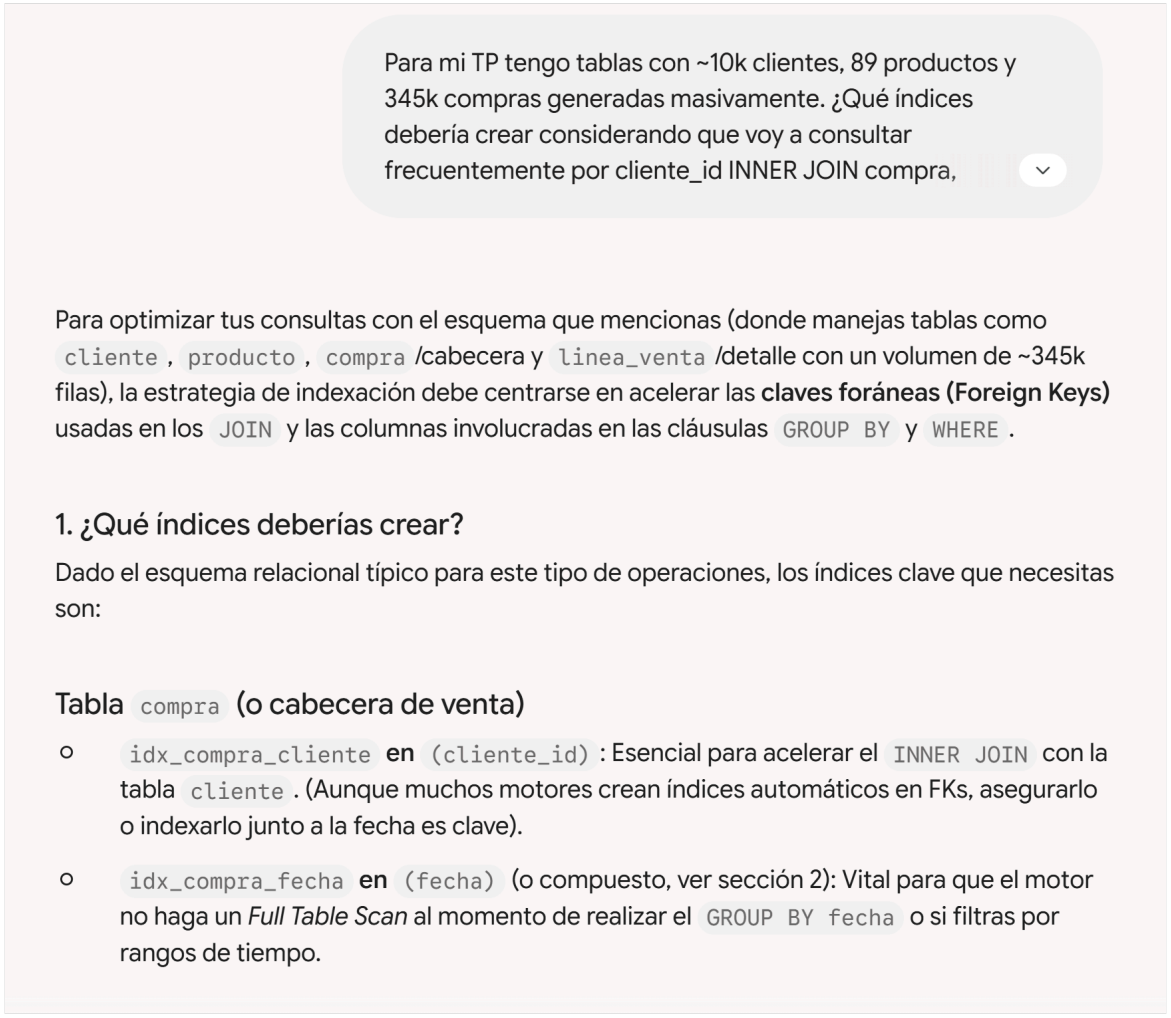


Consulta a ChatGPT sobre distribución sesgada de clientes

8.3 Josue Gallardo — Gemini (Etapas 3 y 5: rendimiento y transacciones)

Consulta 1. Se preguntó qué índices conviene crear en un esquema con decenas de miles de clientes y pedidos y cientos de miles de detalles, considerando JOIN por clave foránea, filtros por fecha y agrupaciones, y si los índices compuestos tienen prioridad sobre los simples.

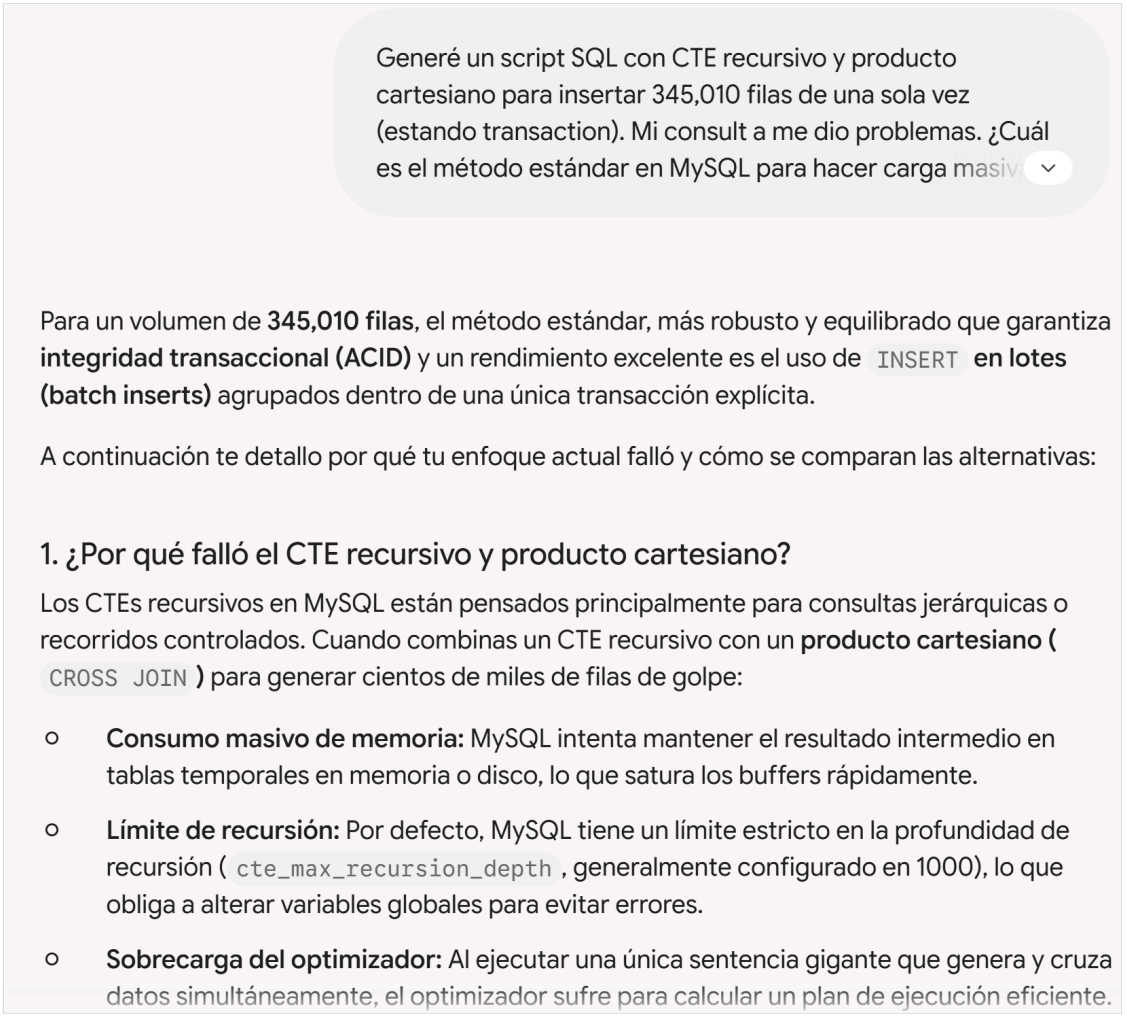
Criterio adoptado. Se confirmó el criterio de indexar las claves foráneas usadas en JOIN y de crear un índice compuesto `(estado, fecha)` para el filtro combinado, que es el que más se repite en las consultas de negocio. Las mediciones de la Etapa 3 muestran la mejora obtenida en igualdad, rango y JOIN.



Consulta a Gemini sobre índices para tablas masivas

Consulta 2. Se preguntó cuál es el método estándar para una carga masiva que garantice integridad transaccional sin perder velocidad: `LOAD DATA INFILE`, INSERT en lote o el CTE recursivo ya implementado.

Criterio adoptado. Se mantuvo el INSERT masivo derivado del CTE dentro de `START TRANSACTION` / `COMMIT`, que resulta equivalente en atomicidad a las otras alternativas y no requiere archivos externos ni permisos adicionales. El mismo criterio transaccional se aplicó luego en `sp_registrar_venta`, con COMMIT y ROLLBACK según el resultado.



Consulta a Gemini sobre carga masiva transaccional

---

9. REFERENCIAS

- Documentación MySQL 8.0: https://dev.mysql.com/doc/refman/8.0/en/
- Consigna TFI Bases de Datos I: [archivo PDF]
- DER original proporcionado por compañero: `DER_reconstruido.md`
- PLAN_TFI.md: Plan de trabajo y reparto del equipo

LINK AL VIDEO DE DEFENSA

[COMPLETAR - Insertar link al video subido a la plataforma]